

From Legal Text to Legal Graphs: Judgment Prediction with Graph Neural Networks

Ziv Barretto

April 26, 2026

Abstract

The growing imbalance between the volume of court cases and the capacity of the judicial system has increased interest in automated methods that can assist legal analysis and outcome prediction. In the Indian setting, this problem is especially challenging because judgments are long, noisy, often multilingual, and frequently spread across multiple hearings. Prediction pipelines using text embeddings risk leaking the final decision through operative text and identity shortcuts. This paper presents a graph embedding based framework for Legal Judgment Prediction over Indian court decisions. The approach constructs a heterogeneous knowledge graph from raw judgments using legal named entities, rhetorical-roles of texts and outcome labels after multi-hearing case consolidation. The approach caters to multilingual documents. By representing cases alongside parties, courts, statutes, provisions, precedents, and argument structures, the model captures both the internal logic of a case and the shared legal relationships across the corpus. Additionally, we introduce a comprehensively annotated dataset of over 73,000 Indian court cases spanning five legal domains, which will be released to support downstream legal NLP research. The proposed framework, using a Heterogeneous Graph Transformer, achieves 80% accuracy and a 0.795 macro-F1 score in the pooled cross-domain setting. The results demonstrate that graph-based representations outperform text-only baselines, offering a robust, interpretable alternative that successfully resists identity-based leakage shortcuts.

Contents

1	Introduction	4
1.1	The Access to Justice Problem in India	4
1.2	What is Legal Judgement Prediction?	4
1.3	Why LJP is Hard	5
1.4	Why Graph-Based Approaches?	5
1.5	Contributions of This Work	6
2	Related Work	7
2.1	Text-Based Legal Judgment Prediction	7
2.2	Graph-Based Approaches	8
2.3	Heterogeneous GNNs and the Motivation for HGT	8
2.4	Indian Legal NLP and OpenNyAI	9
2.5	Gaps This Work Addresses	9
3	System Overview and Data Pipeline	11
3.1	System Architecture Overview	11
3.2	Document Collection and Legal Domains	12
3.2.1	Local Bucketing and Legal Domains	12
3.3	Named Entity Recognition and Rhetorical Role Classification .	14
3.3.1	The OpenNyAI Pipeline	15
3.3.2	Entity Types and Their Graph Roles	15
3.3.3	Rhetorical Role Taxonomy and Leakage-Safe Filtering .	17
3.4	Outcome Labelling with Mistral	18
3.4.1	Labelling Methodology	18
3.4.2	Coverage and Label Distribution	20
3.5	Multi-Hearing Merging	21
3.5.1	The Problem	21
3.5.2	Merge Strategy	22

3.6	Technology Stack	22
4	Heterogeneous Knowledge Graph Construction	23
4.1	Motivation: Why a Heterogeneous Graph?	23
4.2	Graph Schema	23
4.2.1	Node Types	24
4.2.2	Cross-Case Sharing	24
4.2.3	Edges and Graph Structure	25
4.3	Leakage Prevention Strategy	27
4.4	Text Embedding and Feature Extraction	29
4.4.1	The bge-m3 Text Encoder	29
4.4.2	Node Feature Assembly	29
5	Graph Exploration and Structural Insights	31
5.1	Why Explore the Graph Before Prediction?	31
5.2	Graph Statistics and Structural Analysis	32
5.2.1	Overall Graph Statistics	32
5.2.2	Node Type Distribution	32
5.2.3	Degree Distribution and Hub Entities	33
5.2.4	Cross-Domain Bridge Analysis	34
5.2.5	Cross-Bucket Entity Sharing Matrix	34
5.2.6	Connectivity Score Distribution	36
5.3	Per-Domain Corpus Analysis	36
5.3.1	Case Length and Sentence Distributions	36
5.3.2	Entity Richness Per Domain	36
5.3.3	Rhetorical Role Composition Per Domain	37
5.3.4	Outcome Distribution Per Domain	38
5.4	Key Structural Insights	39
6	Prediction Model and Results	40
6.1	Why a Heterogeneous Graph Transformer?	40
6.2	Model Architecture	40
6.3	Experimental Setup	41
6.4	Ablation Variants and Rationale	42
6.5	Baseline Comparisons	43
6.6	Cross-Bucket Results	43
6.7	Per-Domain Results	45
6.8	Ablation Findings	46

6.9	Per-Class Behaviour on the Pooled Set	48
6.10	Best Model: Per-Bucket Per-Class Breakdown	49
6.11	Out-of-Domain Evaluation: A Fully Unseen Legal Domain . .	50
7	Interpretability and Case-Level Explanation	53
7.1	Why Interpretability, and Why at Three Levels	53
7.2	Representation-Level Analysis: What Did the GNN Learn? . .	54
7.3	Component-Level Analysis: Which Parts of the Graph Matter?	56
7.4	Case-Level Explanation	58
7.4.1	Phase 1: Frozen Inference	59
7.4.2	Phase 2: Training a Heterogeneous Edge Masker	59
7.4.3	Phase 3: From Edge Scores to Legal Evidence	60
7.4.4	Phase 4: LLM Translation	60
7.5	A Worked Example: A Confidently-Correct Case	61
7.6	When Confidence and Correctness Diverge: Structural Error Analysis	62
7.7	Summary	63
8	Discussion	65
8.1	Summary of Findings	65
8.2	Limitations	66
8.3	Future Work	67
9	Conclusion	69

Chapter 1

Introduction

1.1 The Access to Justice Problem in India

Access to justice in India is not only a legal ideal but also a large-scale administrative challenge. Court backlogs remain extremely high across the judicial system, which means that delay itself becomes a serious barrier to justice. When cases take years to move, litigation becomes more expensive, evidence becomes harder to trace, and ordinary citizens are less able to sustain meaningful participation in the legal process.

This does not mean that courts should be automated. Judicial decision-making involves interpretation, justification, and public accountability in ways that no prediction system can replace. A more realistic goal is decision support: computational tools that help organise large volumes of case material, surface patterns across past disputes, and assist legal research or case preparation. Legal Judgment Prediction (LJP) sits within that narrower and more defensible ambition.

1.2 What is Legal Judgement Prediction?

Legal Judgment Prediction is the task of estimating some aspect of a court outcome from information contained in the case record [1, 8, 13]. Different papers define the target differently: some predict violation versus non-violation, some predict appeal outcomes, and others predict charges or penalties. In all cases, the core idea is the same: given a representation of a dispute, can a model forecast the judicial result?

In this thesis, LJP is defined narrowly as outcome prediction over Indian court judgments after removing explicit decision-bearing text. The aim is not to generate legal reasoning or normative conclusions, but to test whether pre-decision signals, facts, arguments, statutes, provisions, and precedents, can support outcome prediction when represented in a structured way.

This definition matters because much of the LJP literature has been criticised for hidden leakage [14]. If a model can read operative text, ruling sections, or highly outcome-correlated metadata, then the task becomes answer recovery rather than prediction. For that reason, LJP is as much an experimental-design problem as it is a modelling problem. The central question is not only *what* to predict, but also *what information the model is allowed to use*.

1.3 Why LJP is Hard

LJP is difficult because legal judgments combine four challenges that ordinary classification benchmarks do not face simultaneously: *long documents* that exceed transformer context windows and force truncation or hierarchy [3, 12]; *noisy Indian text* with abbreviations, citation variations, OCR artefacts, transliterated names, and entities that rarely appear in one canonical form, plus the multi-hearing pattern that publishes one dispute as several documents; *relational legal structure* in which outcomes depend on how facts, arguments, statutes, provisions, precedents, and parties connect rather than on isolated sentences; and *leakage*, where decisive phrases such as *appeal allowed* or *petition dismissed*, identity-heavy metadata, or post-decision text can act as shortcuts that recover the outcome rather than predict it. Together these make LJP a representation and evaluation problem first, and only then a modelling problem.

1.4 Why Graph-Based Approaches?

A graph-based approach is a natural fit for legal disputes because cases are structured collections of typed objects and relations. A case is heard in a court, involves parties and lawyers, cites statutes and precedents, and contains argument blocks linked to different sides. When all of this is flattened into one sequence, the model must recover structure that is already present in the

source material. A graph preserves that structure explicitly.

Graphs are useful here for two reasons. First, they preserve heterogeneity: a statute, a party, and a facts section do not play the same legal role and should not be treated as identical inputs. Second, they allow cross-case reasoning through shared legal objects. If two cases cite the same statute or precedent, they are connected in a legally meaningful way, not merely through surface lexical similarity. This thesis therefore argues for a heterogeneous, leakage-audited, cross-case graph representation for Indian LJP.

1.5 Contributions of This Work

This thesis makes five main contributions.

1. It constructs a large structured corpus of Indian court cases spanning five legal domains, with more than 73,000 final-case records after multi-hearing consolidation.
2. It develops an end-to-end pipeline that converts raw judgments into leakage-audited, graph-ready representations using document collection, OpenNyAI enrichment, outcome labelling, multi-hearing merging, and explicit leakage filtering.
3. It proposes a reasoning-focused heterogeneous knowledge graph schema in which cases are represented together with text blocks, parties, courts, statutes, provisions, and precedents, while legally meaningful sharing is separated from identity-based shortcuts.
4. It applies a Heterogeneous Graph Transformer to this graph and evaluates it in the pooled cross-domain setting, showing that structured graph representations outperform text-only baselines on the thesis dataset.
5. It complements prediction with graph analysis, ablations, and leakage-oriented reasoning, treating the graph not just as model input but as an empirical object that helps explain both performance and limitations.

Chapter 2

Related Work

2.1 Text-Based Legal Judgment Prediction

Early LJP relied on bag-of-words and TF-IDF features fed into linear or tree-based classifiers. Aletras et al. predicted ECHR violation judgments with around 79% accuracy from n-gram and topic features over case text, with the facts section emerging as the strongest signal [1], and Katz et al. predicted US Supreme Court outcomes across nearly two centuries with a time-evolving random forest over institutional metadata and voting patterns [8]. Medvedeva et al. then showed that predicting genuinely future cases from past ones substantially lowers performance, exposing temporal leakage as a structural problem in the literature [13], and a subsequent methodological audit found that most LJP publications do not perform genuine prospective prediction at all [14]. The present thesis treats leakage control as a first-class concern for that reason.

The neural wave replaced manual features with learned representations. LEGAL-BERT and the LexGLUE benchmark established that domain-adapted transformers consistently outperform generic ones across legal classification tasks [3, 4]. The persistent weakness of standard transformers on legal text is document length: most judgments exceed 512 tokens and even extended-context architectures incur substantial cost, with hierarchical TF-IDF variants remaining competitive on long inputs [12]. Both feature-engineered and transformer pipelines also share a more fundamental limit: a flat or hierarchical document representation cannot exchange information across cases that share legally meaningful objects. The present thesis treats

text embeddings as node features within a relational representation rather than as the complete model.

2.2 Graph-Based Approaches

Graph-based methods were developed precisely to model the relational structure that flat text pipelines miss. TopJudge formalised dependencies among legal subtasks, law articles, charges, and penalties, as a directed acyclic graph, showing that joint prediction over structured outputs improves performance compared to treating each subtask independently [23]. Subsequent work expanded this relational view by incorporating retrieved precedents and law-article graphs, with more recent systems using LLM-and-retrieval pipelines to bring external legal scaffolding into prediction [22, 19].

Within Indian LJP, graph-based work is growing. Khatri et al. explored GNNs for Indian legal judgment prediction and raised important questions about the role of case-graph design and demographic shortcuts [9]. More recent large-scale Indian work has extended coverage and explored RAG-style pipelines [15]. Existing graph approaches nonetheless leave gaps that this thesis addresses: many are built for civil-law charge-prediction datasets, rely on a single court, or use graph structure to improve prediction without comparable attention to decision-text or identity leakage. The present work is closest in spirit to this line but narrows its claim: the graph should improve prediction by exposing legal structure across cases while explicitly blocking shortcuts that rely on post-decision information.

2.3 Heterogeneous GNNs and the Motivation for HGT

A homogeneous GNN is too blunt for legal graphs: a case node, a statute node, and a precedent node play different epistemic roles, and edges such as *cites_precedent* and *heard_in* should not be collapsed into a single message-passing rule. Among heterogeneous alternatives, HGT [5] fits the design requirements of this thesis best: its node-type-dependent projections and edge-type-dependent attention let the model weight heterogeneous neighbours without the relation-specific weight matrices of R-GCN [20] growing coarse as the type count rises, and without the handcrafted meta-path templates

required by HAN [21]. The graph in this thesis has many node and edge types and a deliberately cross-domain structure, which is exactly the regime HGT was designed for.

2.4 Indian Legal NLP and OpenNyAI

Indian legal NLP developed later than English-language benchmarks for US or European law, in part due to data and tooling scarcity. Early work on rhetorical role identification showed that Indian judgments have a recoverable discourse structure, preamble, facts, arguments, ratio, and ruling, that is useful for downstream tasks but not explicitly marked in raw text [2]. This made legal document structuring a first-class problem rather than a preprocessing convenience.

OpenNyAI is the most practically important development for this thesis, bringing together legal named entity recognition, rhetorical-role prediction, and summarisation for Indian court judgments into an open, modular pipeline [7, 18, 17]. On the benchmarking side, ILDC introduced a large corpus of Indian Supreme Court cases for judgment prediction and explanation [11], while IL-TUR broadened the scope to a multi-task legal understanding and reasoning benchmark [6]. Newer rhetorical-role resources such as LegalSeg confirm that structural modelling continues to matter for Indian judgments [16].

These contributions are substantial, but they do not resolve the core challenge addressed here: how to represent Indian cases so that a model can exploit legal structure across cases without quietly learning identity or post-decision shortcuts. This thesis builds on the Indian legal NLP ecosystem rather than apart from it, combining OpenNyAI’s pipeline components with a leakage-audited heterogeneous graph.

2.5 Gaps This Work Addresses

The literature reviewed above shows that legal judgment prediction has advanced substantially, but along partly disconnected lines. Classical models demonstrated that outcomes are statistically predictable; transformer models improved semantic representation; graph methods showed the value of relational structure; Indian NLP resources made domain-specific preprocessing feasible. Yet most prior work sacrifices at least one of the following: *scale*,

cross-domain coverage, cross-case legal structure, or leakage-aware experimental design.

What remains comparatively rare is an Indian LJP system that operates over a large multi-domain corpus, consolidates multiple hearings per case, extracts legal entities and rhetorical structure, builds an explicitly heterogeneous cross-case graph, and evaluates under a design intended to resist both decision-text leakage and identity-based shortcuts. That is the niche this thesis occupies. Its claim is narrow and, for that reason, defensible: when Indian court judgments are converted into a leakage-audited heterogeneous graph, cross-case legal structure becomes usable in ways that flat text pipelines cannot easily match.

Chapter 3

System Overview and Data Pipeline

This chapter describes the end-to-end pipeline that turns raw Indian court-judgment text into a labelled, leakage-audited, heterogeneous knowledge graph ready for Graph Neural Network (GNN) training. The pipeline was designed around three practical constraints: (i) the corpus is large and noisy, so every stage is a batch process with explicit caching and auditing; (ii) many of the signals a predictor would want (decisions, dispositions, operative paragraphs) are precisely the signals a legally honest experiment must remove, so leakage prevention has to be baked in rather than bolted on; and (iii) the downstream target is a graph, so text extraction and entity resolution have to produce outputs that can be reified as typed nodes and edges without lossy renormalisation later on.

3.1 System Architecture Overview

The system is organised as a six-stage pipeline. Each stage is an independently re-runnable batch process; intermediate artefacts are cached on disk so that an expensive step (OpenNyAI NER on tens of thousands of judgments, or bge-m3 embedding on millions of text spans) never has to be repeated when a later stage changes.

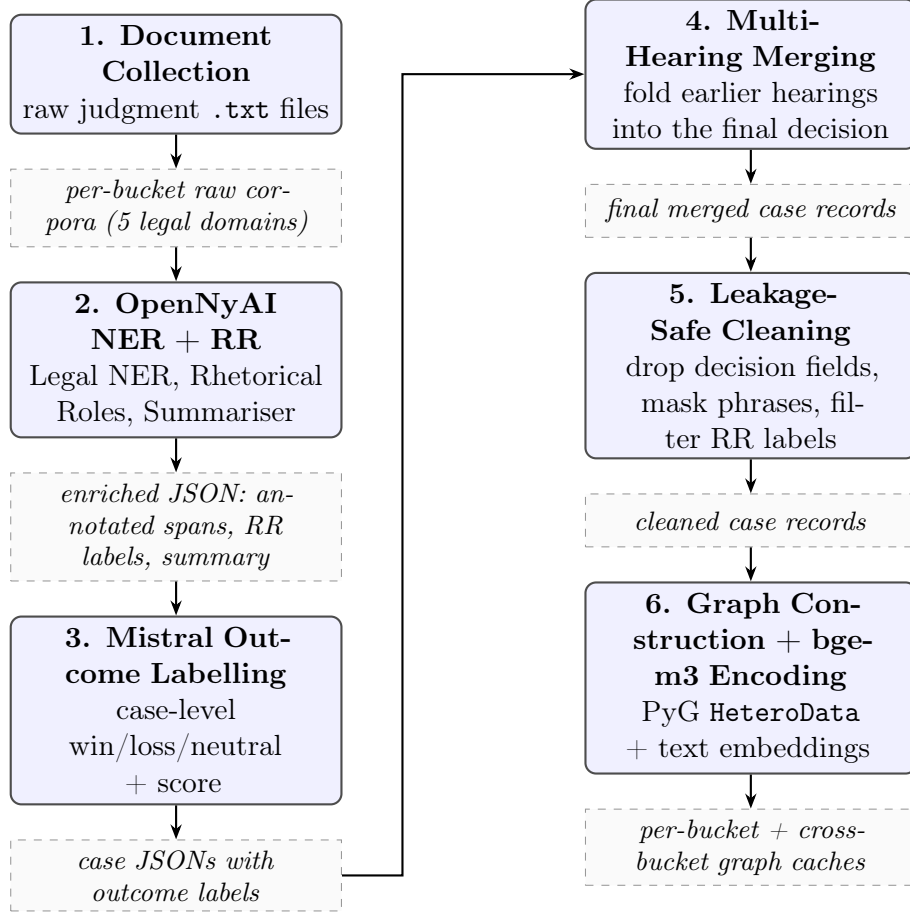


Figure 3.1: The six-stage data pipeline. Solid boxes are processing stages; dashed boxes are cached on-disk artefacts that feed the next stage.

3.2 Document Collection and Legal Domains

3.2.1 Local Bucketing and Legal Domains

Judgments are sourced from Indian Kanoon (indiankanoon.org), whose 40-page-per-query truncation is worked around by issuing court-month-bounded queries so that each window stays within the access limit. Rather than trusting search-engine relevance to define the corpus, which would bias the dataset towards heavily-cited “landmark” cases, domain classification is implemented

locally as a deterministic post-processing step.

After downloading the entire accessible window, each raw PDF underwent text extraction (via PyMuPDF) and was passed through a local filtering script. This script evaluated the text against a rigid set of regular expressions corresponding to required statutory provisions (e.g., “Section 498A IPC”, “Motor Vehicles Act”, “POCSO”) and contextual keyword constraints. Scraped texts that successfully matched a domain’s predefined rules were partitioned into the respective local directory buckets.

The collected corpus resulting from this process is organised into five **legal domain buckets**, chosen so that each bucket is large enough to train a domain-specific graph and internally coherent in terms of statutory framework, party structure, and typical remedies. The buckets are:

- **Family & Matrimonial:** matrimonial disputes, maintenance, custody, and dowry-related proceedings. Dominated by IPC 498A, Sections 482, 506, and the CrPC.
- **Financial Fraud:** cheating, forgery, dishonour, and economic offences. Dominated by IPC 420, 468, 471.
- **Land & Property:** land acquisition, title, and property disputes. Dominated by the Land Acquisition Act 1894 and the Constitution of India.
- **Motor Accidents:** compensation claims and related appeals. Dominated by the Motor Vehicles Act 1988 and Section 166.
- **Sexual Offences:** offences against the person, including POCSO matters. Dominated by the IPC, CrPC, and POCSO Act.

A key design requirement for domain-bucketed training is that no single domain dominates the corpus: a strongly size-imbalanced corpus would confound cross-domain accuracy comparisons, attributing performance differences to sample-size effects rather than to genuine legal-domain structure. Figure 3.2 verifies that the five buckets are broadly comparable after multi-hearing merging, with all domains falling between approximately 14,000 and 15,000 final cases. The near-equal scale means that cross-bucket aggregation is a fair average rather than an average skewed by one large domain.

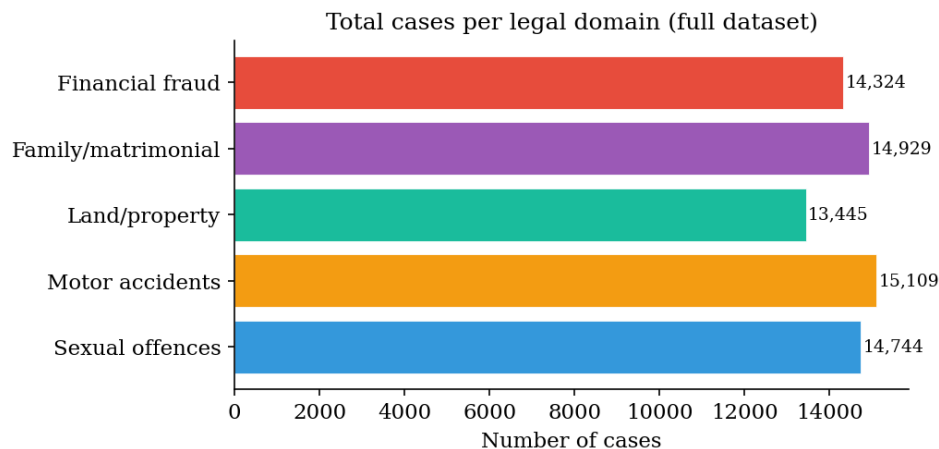


Figure 3.2: Final case counts per legal domain bucket after multi-hearing merging. All five buckets are within a 10% range of each other, confirming that the corpus is balanced at the domain level and that cross-bucket performance comparisons are not confounded by sample-size differences.

3.3 Named Entity Recognition and Rhetorical Role Classification

Before a judgment can enter the knowledge graph, its raw text must be decomposed into typed, structured units: named legal entities tied to precise character positions, a discourse-level rhetorical role assigned to each sentence, and a section-structured summary that separates the preamble, factual background, party arguments, and operative ruling. For this enrichment step we use the OpenNyAI legal NLP stack. The rationale is threefold. First, OpenNyAI’s NER model was trained specifically on Indian court judgments and recognises entity types that a general-purpose tagger misses entirely: statutory provisions cited in the Indian Parliament style, case precedents in the AIR/SCC citation format, and distinct party roles such as PETITIONER, RESPONDENT, and DEFENCE LAWYER. Second, its rhetorical-role classifier is trained on the discourse conventions of Indian High Court judgments, which differ substantially from the conventions of news or scientific text. Third, running the stack locally on GPU gives full control over caching and reproducibility, which is essential when the pipeline may need to be rerun as

the graph schema or leakage-cleaning rules evolve. The following subsections describe the implementation, the output fields produced at each stage, and how those fields feed into graph construction and leakage removal.

3.3.1 The OpenNyAI Pipeline

Three OpenNyAI components are run in sequence for every judgment document.

- **Legal NER** (`en_legal_ner_trf`), configured with sentence-level inference and built-in post-processing, tags named entities by type and normalises them to canonical surface forms.
- **Rhetorical Role Classifier** assigns each sentence one of seven discourse labels reflecting its function within the judgment (PREAMBLE, FACTS, ARGUMENTS, RATIO, RPC, RLC, and NONE).
- **Summarizer**, conditioned on the predicted rhetorical roles, produces a section-structured extractive summary with four named blocks: PREAMBLE, FACTS, ARGUMENTS, and DECISION.

The environment is pinned (spaCy 3.2.4, pydantic 1.7.4, CUDA-enabled PyTorch and CuPy). Each input judgment produces a single combined JSON record that is extended by every subsequent pipeline stage.

The subsections that follow describe the semantic role of the fields these components produce, and how they are consumed downstream.

3.3.2 Entity Types and Their Graph Roles

OpenNyAI’s legal NER recognises twelve entity types; the subset reified as graph nodes in the downstream knowledge graph is:

- **STATUTE** and **PROVISION**: statutes (e.g. the IPC, the CrPC, the Motor Vehicles Act) and their sections (e.g. Section 482). Provisions are additionally linked to their parent statute via a `belongs_to_statute` edge.
- **PRECEDENT**: cited case references.

- COURT and JUDGE: the court the matter is heard in and the presiding bench.
- PETITIONER, RESPONDENT, PETITIONER_LAWYER, DEFENCE_LAWYER, LAWYER: typed party and counsel roles. Role assignment uses the section in which the mention first appears (preamble versus argument blocks) combined with OpenNyAI’s entity labels.
- ORG, GPE, DATE, CASE_NUMBER: auxiliary types extracted by NER, retained in the cleaned JSON but excluded from the final reasoning-focused graph (Section 4.2).

Entities are canonicalised per bucket: surface forms are normalised (case, whitespace, punctuation), grouped, and re-projected as `canonical_name`. Canonicalisation allows the same statute or court to appear as a *single shared node* across the thousands of cases that mention it, which is the structural property that makes the graph useful for cross-case reasoning.

To verify that the extraction stage is producing entity distributions consistent with the legal character of each domain, Figure 3.3 summarises the total entity mention counts by type across the full corpus. The figure shows that precedents dominate in absolute volume, confirming that citation is the primary relational artefact in Indian court judgments. Party nodes (PETITIONER and RESPONDENT) are large and roughly equal in volume across the corpus as a whole, reflecting the adversarial structure of litigation. Statutes and provisions, by contrast, have comparatively lower raw mention counts but are the node types that carry the most cross-case structural weight: each canonicalised mention is a potential shared edge linking the current case to every other case that cites the same authority. This asymmetry between volume and structural importance is what motivates treating statutes, provisions, and precedents as the shared layer of the graph while keeping party nodes local to each case.

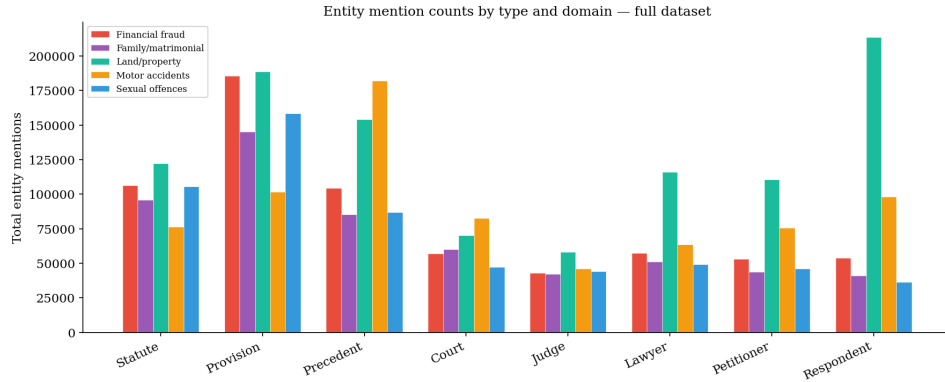


Figure 3.3: Total entity mention counts by type across the full corpus after NER and canonicalisation. Precedent mentions dominate in volume; statutes and provisions are smaller in count but form the shared-node layer that carries structural weight across cases.

3.3.3 Rhetorical Role Taxonomy and Leakage-Safe Filtering

OpenNyAI’s rhetorical-role taxonomy is the hinge on which the leakage-safety story turns. Of the seven labels produced by the classifier, two are inherently decisive: **RPC** (Ratio and Precedent Cited, in practice the court’s operative paragraph) and **RLC** (Ruling by Lower Court). Keeping either of them in the training data would let a classifier read the outcome directly from the text.

The cleaning stage therefore treats both labels as forbidden annotation labels: any annotation whose label set intersects $\{\text{RPC}, \text{RLC}\}$ is dropped outright, along with any annotation whose summary section is **DECISION** or whose label string contains "**decision**", "**disposition**", "**operative**", "**order**", or "**rpc**". Only the three non-decisive summary fields (**PREAMBLE**, **FACTS**, and **ARGUMENTS**) are retained as text sources for the graph’s text nodes. This is the first leakage gate; a second, phrase-level gate is described in Section 4.3.

3.4 Outcome Labelling with Mistral

With judgments enriched by NER and rhetorical-role annotations, the next requirement is a case-level outcome label: did the appellant win, lose, or receive only a procedural disposition? We produce these labels automatically by prompting a Mistral large language model with the section-structured summaries that OpenNyAI already generates. This approach has two important properties. First, the model is shown only the `DECISION` summary block and the `RPC` sentences, which are precisely the portions that the leakage-cleaning stage will subsequently remove. The labeller therefore never sees the non-decisive preamble, facts, or arguments text that the GNN will train on, eliminating any circularity between what gets labelled and what gets learned. Second, the section-structured input provides a compact, well-delimited context that substantially reduces the chance that the model grounds its judgement on spurious signals from the surrounding narrative.

3.4.1 Labelling Methodology

Labels. Three outcome categories are defined from the perspective of the party seeking relief (the appellant, petitioner, or applicant):

- `appellant_won` (`score = +1`): relief was granted, the petition or appeal was allowed, bail was granted, a conviction was quashed, or an order was set aside.
- `appellant_lost` (`score = -1`): the petition, appeal, or application was dismissed, rejected, or denied; bail was refused; or relief was withheld.
- `postponed_or_procedural` (`score = 0`): the matter was adjourned, notice was issued, a liberty was granted to pursue another remedy, the case was remanded, or the outcome cannot reasonably be treated as a final win or loss.

For each judgment the model also returns a `confidence` rating (`high`, `medium`, or `low`) and a short textual explanation, both stored alongside the label for auditing purposes.

Cross-validation design. LLM labels are sensitive to run-to-run sampling variance, particularly on procedural dispositions where small wording shifts can flip `postponed_or_procedural` into `appellant_lost`. To bound that noise, a second independent labelling pass was run with a different prompt design: instead of a single free-form label the model answers eight binary yes/no questions (Table 3.1), six directional win/loss signals plus a two-question neutrality clause that overrides them when fired. The final label is then computed by a deterministic aggregation over the binary votes, so the cross-validation pass eliminates the free-text generation variance of the first pass.

Group	ID	Question
A: Win signals	Q1	Did the appellant win the case?
	Q2	Was the petition, appeal, or application allowed?
	Q3	Did the court rule in favour of the petitioner or appellant?
B: Loss signals	Q4	Did the respondent win the case?
	Q5	Was the petition, appeal, or application dismissed?
	Q6	Did the court rule against the appellant?
C: Neutrality clause	Q7	Was the case adjourned, remanded, or deferred without a final decision?
	Q8	Is there no clear final judgment in this case?

Table 3.1: The eight binary yes/no questions posed to the model in a single call during the cross-validation pass. Questions Q1–Q6 are the directional win/loss signals; Q7–Q8 form the neutrality clause that overrides the directional evidence when fired.

Aggregation sums the directional answers into $S_{\text{win}} = q_1 + q_2 + q_3$, $S_{\text{loss}} = q_4 + q_5 + q_6$, and $S_{\text{neutral}} = q_7 + q_8$, with a fixed priority: a non-zero neutrality clause yields `postponed_or_procedural`; otherwise the majority directional signal wins, ties resolve to procedural, and an all-zero response is marked `unknown` and excluded from training.

Results. Across the 72,795 files for which both labels exist, the two passes agree on **84.4%** of cases (Table 3.2).

Bucket	Files compared	Matching labels	Agreement rate
Family & Matrimonial	15,017	13,049	86.9%
Financial Fraud	14,342	12,015	83.8%
Land & Property	13,512	9,652	71.4%
Motor Accidents	15,117	13,552	89.6%
Sexual Offences	14,807	13,143	88.8%
Total	72,795	61,411	84.4%

Table 3.2: Cross-run label agreement between the original Mistral pass and the binary-question cross-validation pass. Agreement is computed per-file over files present in both runs.

Disagreement is concentrated on procedural dispositions, which the second pass tends to reclassify as APPELLANT_LOST. The effect is sharpest in *Land & Property* (71.4% agreement), where remands and stay orders read as adverse without full context. Label noise is therefore real but bounded and disproportionately attached to procedural cases; the downstream GNN excludes the POSTPONED_OR_PROCEDURAL class and trains on the binary APPELLANT_WON vs. APPELLANT_LOST subset, on which agreement is materially higher.

3.4.2 Coverage and Label Distribution

Labelling coverage is near-complete across all five buckets; the small unlabelled fraction consists of cases where Mistral returned a malformed response or an explicit `unknown` verdict, which are excluded from training rather than assigned a default label. Figure 3.4 shows the resulting per-bucket win/loss/neutral rates. Motor-accidents is the most win-skewed bucket, compensation claims are allowed more often than denied at the appellate stage, while financial-fraud and sexual-offences sit closest to balanced. The pooled cross-bucket win rate of roughly 60% WIN to 40% LOSS (on the binary subset, once procedural cases are excluded) is what motivates reporting macro-F1 alongside accuracy in Chapter 6: a constant-win classifier would otherwise achieve misleadingly high accuracy while failing systematically on the loss class.

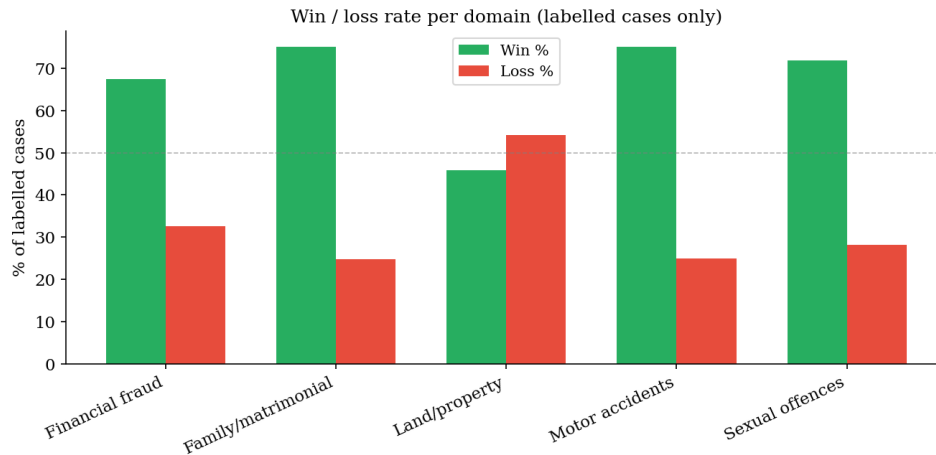


Figure 3.4: Outcome label rates (proportions) per bucket. Motor-accidents is the most appellant-favourable domain; financial-fraud and sexual-offences are closest to balanced.

3.5 Multi-Hearing Merging

3.5.1 The Problem

A single legal matter frequently produces more than one judgment document: interim orders, remands, partial disposals, and the final decision may all be published as separate PDFs with distinct Indian-Kanoon-style filenames that nonetheless refer to the same case. If these hearing files are fed into the graph independently, three things go wrong:

1. duplicate CASE nodes for what is actually one dispute inflate the node count and double-count outcomes;
2. the outcome label of the final hearing is not propagated to the earlier hearings, so the training signal is noisy; and
3. cross-case structural signals (shared parties, statutes, and counsel across hearings) are double-counted as if they were independent observations.

3.5.2 Merge Strategy

Documents that share both a case title *and* the court-assigned case registration number (e.g. 5941/2024, Crm-M-44633-2024) extracted from the preamble are grouped as the same proceeding; the outcome label of the latest hearing is propagated to the merged record, and earlier hearings whose text is already subsumed by the latest one are suppressed. Title alone is insufficient because two unrelated litigants can share a name (e.g. “*Amit Kumar vs State of Bihar*” on different dates can refer to entirely separate matters). FIR numbers are deliberately not used as a merge key, since the same FIR can give rise to independent proceedings in different courts.

The resulting fold is non-trivial but concentrated:

Bucket	Inputs	Final cases	Folded in	Appended	Already contained
Family & Matrimonial	15,800	15,306	493	490	3
Financial Fraud	15,014	14,615	399	396	3
Land & Property	14,324	14,051	251	242	9
Motor Accidents	15,271	15,236	32	30	2
Sexual Offences	15,532	14,596	936	932	4
Total	75,941	73,804	2,111	2,090	21

Table 3.3: Multi-hearing merge statistics. 2,111 older hearing documents were folded into 2,090 merged final-case JSONs; the remaining 21 were already fully subsumed by the latest hearing text.

3.6 Technology Stack

The pipeline is Python-based throughout: OpenNyAI for NER, rhetorical roles, and summarisation; a batched Mistral client for outcome labelling; PyTorch Geometric’s `HeteroData` for graph construction; and BAAI/bge-m3 for text embeddings.

Chapter 4

Heterogeneous Knowledge Graph Construction

4.1 Motivation: Why a Heterogeneous Graph?

Each cleaned case JSON from Chapter 3 already carries typed entities, rhetorical-role-segmented text, and an outcome label. A flat document representation discards two things this structure provides. It collapses typed objects, a court, a statute, a provision, a party, into a single token stream that a model has to re-disambiguate from scratch, and it has no mechanism to exchange information *between* cases that share the same statute or precedent: when two cases both cite IPC 498A, a flat view sees the string twice while a graph view sees two cases attached to the same shared node, where one round of message passing can move information from one to the other. A heterogeneous graph captures both properties by design: distinct node and edge types preserve typed semantics, and shared entity nodes provide cross-case conduits. The rest of this chapter describes the schema, the leakage-prevention strategy, and the text-embedding features attached to each node.

4.2 Graph Schema

The graph is constructed as a **full-star global graph**: every case is a local "star" of text and entity nodes anchored on a CASE node, and a selected set of entity types are *shared* across cases so that stars of different cases are stitched into one connected component.

4.2.1 Node Types

The reasoning-focused graph keeps **17 node types** in three categories:

- **The case node.** A single CASE node per case, whose attributes store the outcome label, bucket, filing and decision dates (used for temporal gating), and bookkeeping metadata.
- **Text nodes.** Six types – PREAMBLE, FACTS, ARGUMENTS, PETITIONER_ARGUMENTS, RESPONDENT_ARGUMENTS, and OTHER_LAWYER_ARGUMENTS – each carrying a `bge-m3` text embedding of the cleaned, leakage-filtered span they represent (Section 4.4).
- **Entity nodes.** Ten types: PETITIONER, RESPONDENT, PETITIONER_LAWYER, DEFENCE_LAWYER, LAWYER, COURT, JUDGE, STATUTE, PROVISION, and PRECEDENT.

Four auxiliary NER types that OpenNyAI produces – ORG, GPE, DATE, and CASE_NUMBER – are deliberately omitted from the reasoning-focused graph. They carry little predictive signal for the outcome and would otherwise inflate the node count and introduce weak, noisy edges that dilute message passing.

4.2.2 Cross-Case Sharing

Whether two cases share a node, or each hold a private copy of one, determines whether a GNN can use that node as a cross-case conduit. The policy is deliberate:

- **Shared across cases:** STATUTE, PROVISION, and PRECEDENT. These are the *law itself* – citing IPC 420 in one case and IPC 420 in another is meant to be the same object, and sharing the node is what lets the GNN propagate information along the citation.
- **Local to each case:** everything else, including COURT, JUDGE, and all LAWYER variants. These are *identity proxies*: if a particular judge or a particular bench has a historical win/loss bias, sharing the node would let the GNN recover that bias as a free shortcut to the outcome – exactly the kind of spurious signal the experimental design in Chapter 6 is built to prevent. Keeping them local makes the model reason about the law, not about the docket.

4.2.3 Edges and Graph Structure

Edges in the schema fall into three groups, summarised in Table 4.1: *structural* edges anchoring text and entity nodes to their parent CASE, *citation* edges forming the legal backbone, and *counsel–argument* edges tying each lawyer and party to their own argument block. Citations attach to the ARGUMENTS node rather than the case so that a reference sits in the discourse context of the side that invoked it, and the provision hierarchy is preserved by PROVISION–STATUTE edges so a citation to Section 482 sits two hops from the IPC rather than disconnected from it. Every edge is added in both directions so HGT’s message passing can deliver neighbour information back to the case representation that the classifier reads.

Group	Edges
Structural	CASE–text (preamble / facts / arguments / petitioner / respondent / other-lawyer arguments); CASE–party; CASE–court; CASE–judge; CASE–lawyer (each role).
Citation	ARGUMENTS–STATUTE; ARGUMENTS–PROVISION; ARGUMENTS–PRECEDENT; PROVISION–STATUTE hierarchy.
Counsel / party	PETITIONER_LAWYER–PETITIONER_ARGUMENTS; DEFENCE_LAWYER–RESPONDENT_ARGUMENTS; LAWYER–OTHER_LAWYER_ARGUMENTS; party–matching-argument-block.

Table 4.1: Edge groups in the reasoning-focused schema. Every edge is added in both directions.

Figure 4.1 illustrates the local “star-graph” structure surrounding a single CASE node, with the citation edges from the ARGUMENTS node fanning out to shared statutes, provisions, and precedents. These citation edges are constrained chronologically: a precedent edge is added only when $\text{precedent_year} < \text{case_year}$, and a statute edge only when $\text{statute_year} \leq \text{case_year}$, so a case can never structurally link forward to authorities that did not exist at the time it was decided.

Figure 4.2 shows how those shared nodes connect multiple cases into one graph. Statutes, provisions, and precedents act as cross-case conduits along which the GNN can propagate information; courts, judges, and individual

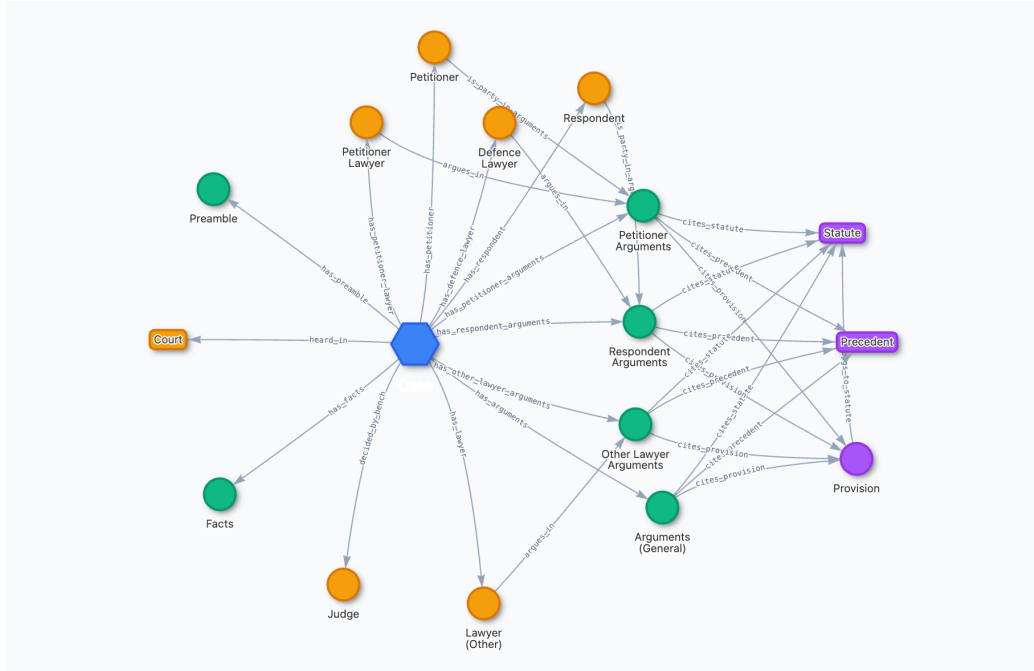


Figure 4.1: Star-graph structure for a single CASE node. Text blocks attach as text nodes; identity entities (COURT, JUDGE, LAWYER) attach as local entity nodes. The ARGUMENTS node carries the citation backbone out to shared STATUTE and PRECEDENT nodes.

lawyers are kept private to each case so that docket-level identity cannot leak the outcome.

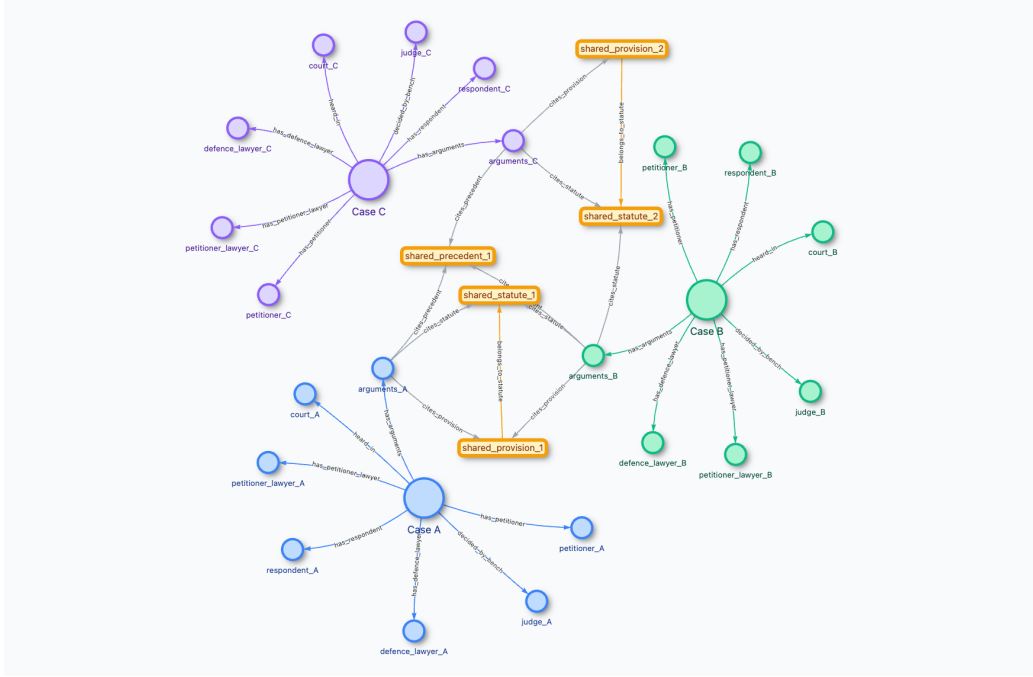


Figure 4.2: Cross-case connection through shared legal authorities. Identity entities remain local; statutes, provisions, and precedents form the cross-case conduits used for message passing.

Each CASE node carries its outcome label, bucket membership, and the dates needed for temporal gating; text nodes carry their **bge-m3** embedding (Section 4.4); entity nodes carry their canonical name. Edges are typed with their relation, which is what the HGT layers use to index their per-relation parameters (Chapter 6).

4.3 Leakage Prevention Strategy

Legal judgment prediction has a notorious failure mode: models that appear to predict the outcome are in fact *reading* it. A graph built straight from the OpenNyAI output would expose the verdict through several distinct channels,

top-level outcome fields written by the labeller, decisive rhetorical-role spans (RPC, RLC), residual decision phrases buried inside otherwise non-decisive sentences, and shared PRECEDENT or STATUTE nodes that link a case to authorities decided after it. Each of these is closed off explicitly before the graph is materialised.

Field-level removal. The cleaning stage strips every top-level field that contains, scores, or paraphrases the outcome, the case outcome label, the outcome score, the labeller’s confidence and short explanation, the raw model response, the extracted decision text, and the cached RPC sentences, along with the DECISION block of the OpenNyAI summary. After this step no case record retains a field that names its own verdict.

Rhetorical-role filtering. The two decisive role labels, RPC (the court’s operative paragraph) and RLC (the ruling by the lower court), are dropped together with any annotation whose label string matches a small blacklist (decision, disposition, operative, order, rpc). This is the single most important gate: without it the ARGUMENTS text node would contain the verdict verbatim.

Phrase-level masking. Decisive language can survive role filtering when it is embedded inside a FACTS or ARGUMENTS sentence rather than tagged as RPC. A curated list of outcome-bearing phrases, “*petition stands disposed of*”, “*appeal is allowed*”, “*set aside*”, “*quashed*”, and so on, is therefore applied as a case-insensitive, whitespace-tolerant scrubber across every retained text field, replacing each match with a [LEAKAGE_MASK] token.

Temporal gating on shared entities. A shared PRECEDENT or STATUTE node would leak future information if a citation edge were attached to a case decided before the cited authority existed. The graph builder enforces a strict rule: a precedent edge is added only when precedent_year < case_year, and a statute edge only when statute_year ≤ case_year. Cross-case information therefore flows only along legally honest paths.

Audit trail. Every case carries a leakage_audit record listing the fields dropped, annotations removed, decision text removed, and phrase-level masks applied. The audit is stored alongside the graph cache so that any future

claim of leakage can be checked against the per-case ledger rather than argued in the abstract.

Together these gates leave the GNN with a strictly pre-decision view of the dispute, the parties, the facts, what each side argued, and the law they invoked, with the verdict itself systematically withheld.

4.4 Text Embedding and Feature Extraction

4.4.1 The bge-m3 Text Encoder

All text nodes are featurised with BAAI/bge-m3, a multilingual sentence-transformer with a 1024-dimensional output, run through the `sentence_transformers` library. The choice of bge-m3 is driven by two concrete requirements: (i) Indian legal text mixes English with Hindi and regional-language transliterations, Latin-script legal abbreviations, and citation fragments, and bge-m3’s multilingual training data handles this noticeably better than English-only encoders; and (ii) the model is strong in retrieval settings, which aligns with the graph’s use of embeddings as comparison features across cases.

Configuration. The encoder is run with `batch_size=16`, `max_length=512`

Hierarchical chunking for long spans. Legal judgment *Facts* and *Arguments* sections routinely exceed the model’s 512-token window. For texts longer than the configured character limit, the encoder splits them into overlapping, sentence-boundary-aligned chunks, encodes each chunk independently, and mean-pools the resulting vectors. This preserves a consistent 1024-dimensional representation per text node regardless of input length, while avoiding the information loss of a hard truncation.

Caching. Embeddings are cached per namespace with a deterministic hash of the input ids, texts, and encoder configuration.

4.4.2 Node Feature Assembly

For each node type the final per-node feature vector is the concatenation of:

- the 1024-dim `bge-m3` embedding of the node’s canonical text (for text nodes, that is the full span; for entity nodes, that is the canonical name);
- scalar node-level features: local frequency in the case, section-of-first-mention indicators (`seen_in_preamble`, `seen_in_arguments`), type one-hot, and for shared entity types the log global frequency across the corpus;
- for the CASE node, bucket one-hot and the outcome label is attached only as the supervision target, never as a feature.

Because different node types carry different feature dimensionalities, the HGT model used in Chapter 6 begins with a per-node-type linear projection (`input_projections`) that maps all features into a common hidden dimension before the first heterogeneous message-passing layer. This is the mechanism that lets the graph remain genuinely heterogeneous at the feature level while still admitting a uniform message-passing computation.

Chapter 5

Graph Exploration and Structural Insights

5.1 Why Explore the Graph Before Prediction?

It is tempting to treat the graph built in Chapter 4 as a black box that is fed into a GNN and evaluated by a single F1 number. That view is wrong for two reasons.

First, *the graph is the dataset*. In a flat-text pipeline the dataset is a set of labelled documents; here the dataset is a typed, relational object with cross-domain structure, and any claim a later chapter makes about the model’s behaviour presupposes a concrete understanding of that structure. Whether the model’s accuracy is driven by genuine cross-case reasoning or by a few dominant hub statutes, whether the five legal domains are actually five or mostly one, whether the per-case subgraph has the connectivity a message-passing layer needs – none of these questions can be answered by looking at prediction results alone.

Second, *leakage-safety claims are structural claims*. The leakage prevention strategy in Section 4.3 is only as convincing as the structural evidence that the remaining graph is still dense enough to support learning. If removing RPC annotations, decision fields, and identity-proxy shortcuts left behind a near-disconnected graph, the downstream accuracy would be either implausibly high (still leaking) or implausibly low (now uninformative). The exploration stage is where we verify that neither is the case.

This chapter therefore treats the graph as an object of study in its own

right. Section 5.2 summarises the overall structure; Section 5.3 breaks it down per legal domain; Section 5.4 distils the conclusions that feed forward into Chapter 6.

5.2 Graph Statistics and Structural Analysis

5.2.1 Overall Graph Statistics

The final reasoning-focused graph, assembled across all five legal domain buckets, contains **880,176 nodes** and **1,977,295 edges**. The average node degree is **4.49**; the maximum degree is **29,602**, attained by the IPC statute node.

At the per-bucket level (before cross-bucket sharing is expanded into the unified view the GNN trains on), the five buckets have broadly comparable case counts but markedly different internal densities:

Bucket	Cases (files)	Nodes	Edges
Family & Matrimonial	15,307	97,150	3,660,068
Financial Fraud	14,616	113,240	7,869,952
Land & Property	14,052	126,962	9,210,612
Motor Accidents	15,237	125,234	7,165,461
Sexual Offences	14,597	106,662	7,123,225
Cross-bucket unified	73,809	485,540	27,243,141

Table 5.1: Per-bucket and cross-bucket graph statistics. The cross-bucket graph is denser than the sum of the per-bucket graphs because shared STATUTE, PROVISION, and PRECEDENT nodes acquire edges from every bucket that cites them.

Family-matrimonial is the sparsest bucket by edges-per-case (≈ 239), and land-property is the densest (≈ 655); the difference is driven mostly by the number of distinct provisions and precedents cited per case in each domain.

5.2.2 Node Type Distribution

The node-type breakdown of the unified graph is strongly skewed towards **parties and precedents**:

Node type	Count	Share
CASE	72,735	8.3%
PRECEDENT	255,148	29.0%
RESPONDENT	192,147	21.8%
PETITIONER	171,084	19.4%
LAWYER	73,645	8.4%
PROVISION	43,504	4.9%
COURT	38,799	4.4%
STATUTE	21,754	2.5%
JUDGE	11,360	1.3%

Table 5.2: Distribution of the 880,176 nodes by type in the unified reasoning-focused graph.

5.2.3 Degree Distribution and Hub Entities

The graph is strongly hub-dominated. The top-12 nodes by degree are all statutes, provisions, or courts – exactly the node types the schema shares across cases:

Node	Type	Degree	Buckets bridged
IPC	Statute	29,602	5
CR.P.C.	Statute	18,704	5
Supreme Court	Court	17,294	5
Apex Court	Court	14,242	5
Indian Penal Code	Statute	12,009	5
High Court of Judicature at Allahabad	Court	8,833	5
Section 482	Provision	8,732	5
Section 506	Provision	8,258	5
I.P.C.	Statute	7,051	5
Section 420	Provision	6,285	5
Section 323	Provision	6,002	5
Constitution of India	Statute	5,950	5

Table 5.3: Top hub entities by degree, with the number of buckets they bridge. Every one of the top 12 hubs appears in all five legal domains.

That every one of the top hubs bridges all five buckets is not an accident:

these are the statutes and courts every Indian criminal and civil proceeding touches, and they are where cross-case message passing actually happens.

The degree distribution is heavy-tailed and scale-free-like, so cross-case reasoning capacity is structurally concentrated in a handful of statute and court nodes; the connectivity score in Section 5.2.6 quantifies how well each individual case is wired into that hub layer.

5.2.4 Cross-Domain Bridge Analysis

The buckets are not isolated. Table 5.3 already shows that the top hubs are bridges by construction; the full bridge analysis confirms that the connective tissue between domains is dominated by the general criminal-procedure and penal-code statutes, with domain-specific bridges layered on top:

- IPC, CR.P.C., INDIAN PENAL CODE, CONSTITUTION OF INDIA, and SUPREME COURT / APEX COURT appear in all five buckets and together account for the majority of cross-bucket edges.
- Domain-specific statutes bridge fewer buckets but at higher within-bucket intensity: POCSO ACT appears in four buckets but is heavily concentrated in sexual-offences; MOTOR VEHICLES ACT, 1988 appears in all five but is concentrated in motor-accidents; LAND ACQUISITION ACT, 1894 is concentrated in land-property.
- Provisions like Section 482, 506, 420, and 323 are multi-bucket because they sit in the CrPC and IPC, which are themselves multi-domain.

5.2.5 Cross-Bucket Entity Sharing Matrix

Knowing which individual nodes are hubs is useful; knowing how many entities are shared *between each pair of domains* is a different and complementary question. If two domains share very few entities, cross-domain transfer between them is structurally impossible, there are no shared nodes for message passing to flow through. If two domains share nearly all entities, they are functionally the same graph and a separate bucket structure adds no value. The cross-bucket sharing matrix in Figure 5.1 quantifies this pairwise sharing at the canonicalised entity level across the full graph.

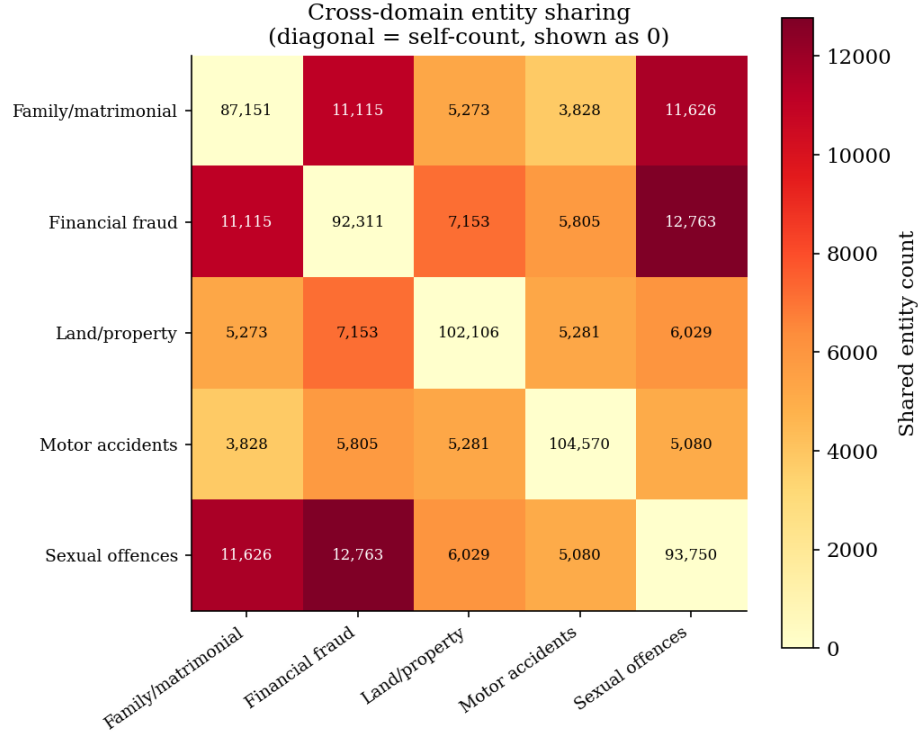


Figure 5.1: Cross-bucket entity sharing matrix for the full graph. Each cell shows the number of canonicalised entities shared between two legal domain buckets. Darker values indicate more sharing. The three IPC/CrPC-driven buckets, family-matrimonial, financial-fraud, and sexual-offences, share heavily with each other; land-property is more weakly connected to the rest via its distinct statutory backbone.

Three patterns are visible in the matrix:

1. Strong pairwise sharing among *Family & Matrimonial*, *Financial Fraud*, and *Sexual Offences*, all three are IPC/CrPC-driven and share the same high-degree statute and provision nodes.
2. Moderate sharing between those three and *Motor Accidents*, mediated by the general procedural statutes and by the Supreme Court and High Court nodes that every appellate domain shares.
3. Comparatively weaker sharing with *Land & Property*, which is anchored by its own statutory backbone (Land Acquisition Act 1894, Constitution

of India) and consequently has fewer canonical entities in common with the criminal-procedure-dominated domains.

This pattern is exactly what a cross-domain transfer claim needs to be credible. The domains are related enough that a shared graph representation should carry useful signal across all five buckets; they are different enough that each bucket retains its own statutory fingerprint and that a single-domain classifier should not trivially dominate a cross-domain one. The moderate but not extreme sharing level motivates the design choice of a unified cross-bucket graph rather than five separate per-domain graphs: there is sufficient shared structure to benefit from joint training, but sufficient domain separation to make that benefit non-trivial.

5.2.6 Connectivity Score Distribution

The hub analysis above characterises individual nodes; a complementary question is how well each *case* as a whole is wired into the shared-entity layer. A bucket-normalised **connectivity score** captures this for every case: cases that cite many hub statutes and widely-cited precedents score high; cases that sit primarily on rare authorities score low.

5.3 Per-Domain Corpus Analysis

The structural view above aggregates across buckets; the per-domain view is where the legal character of each bucket shows up.

5.3.1 Case Length and Sentence Distributions

Case length varies substantially across buckets and directly drives the `bge-m3` chunking strategy of Section 4.4.

5.3.2 Entity Richness Per Domain

Entity richness, the number of unique canonicalised entities per case, and mean case degree are both per-bucket structural properties of the graph.

The PageRank ranking within each bucket makes the domain fingerprint concrete:

- **Family & Matrimonial:** IPC, CrPC, Sections 498A, 482.
- **Financial Fraud:** IPC, CrPC, Sections 420, 468, 471.
- **Land & Property:** Supreme Court, Land Acquisition Act 1894, Constitution of India.
- **Motor Accidents:** Motor Vehicles Act 1988, Supreme Court, Apex Court, Section 166.
- **Sexual Offences:** IPC, CrPC, POCSO Act, Section 506.

The top-5 per bucket is a near-perfect fingerprint of the underlying legal domain, which is a reassuring sanity check: the canonicalisation and the NER are recovering legally meaningful primary citations, not surface noise.

The ranked lists tell us *which* entities are important, and two structural patterns feed forward into the prediction chapter: domains with spread, multi-centred geometry (land-property, financial-fraud) carry more diversity for multi-hop message passing to exploit than domains with tight, low-diversity backbones (motor-accidents); and a three-tier bridge structure, universal apex-court bridges, mid-tier IPC/CrPC bridges spanning the criminal-procedure domains, and narrow domain-exclusive authorities, gives the heterogeneous graph enough shared structure for cross-domain representation learning while preserving per-domain fingerprints.

5.3.3 Rhetorical Role Composition Per Domain

The rhetorical-role composition of the corpus is what tells us whether the leakage gate from Chapter 3 actually fired. The two decisive labels, RPC (the operative paragraph) and RLC (the lower-court ruling), with ANALYSIS and the RATIO / PRECEDENT-RELIED class also carrying outcome signal, have to be absent from every bucket once cleaning is applied. Figure 5.2 shows the per-bucket role distribution side-by-side, before filtering on the left and after filtering on the right.

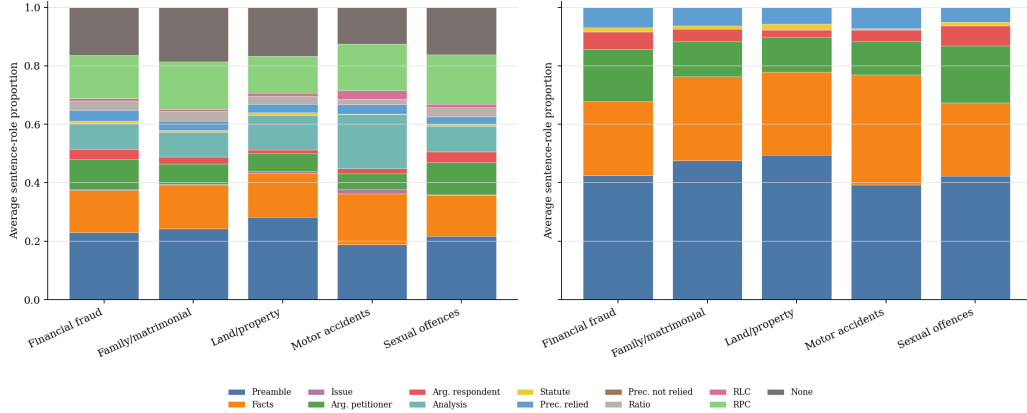


Figure 5.2: Rhetorical role composition per bucket before (left) and after (right) leakage-safe filtering. The pre-filter view contains the full taxonomy, including RPC, RLC, RATIO, ANALYSIS, and the precedent-relied / not-relied roles. The post-filter view retains only PREAMBLE, FACTS, petitioner and respondent arguments, and the precedent-relied trace.

The contrast between the two panels is the structural confirmation that the leakage filter is doing what it is supposed to. On the left, every bucket carries a visible RPC, RLC, and ANALYSIS share, together with a NONE-class residual. On the right, those decisive bands are gone in every bucket, the post-filter composition collapses onto PREAMBLE, FACTS, and the two argument streams, with only the precedent-relied trace retained as non-decisive citation context. The remaining FACTS and ARGUMENTS mass is what the GNN actually trains on, and its relative size still tracks case length: land-property and financial-fraud carry the largest ARGUMENTS share, motor-accidents the smallest, consistent with the sentence-count ordering noted earlier in this chapter.

5.3.4 Outcome Distribution Per Domain

The outcome label distribution and its per-domain win/loss rates were analysed in detail as part of the pipeline audit in Chapter 3 (Figure 3.4). From the graph-exploration perspective, the relevant structural observation is that the class imbalance is *domain-dependent*: motor-accidents is the most win-skewed, financial-fraud and sexual-offences sit closest to balanced. This means the

GNN’s training signal is not uniformly imbalanced, different domains present different prior odds, which in turn means that per-domain calibration, rather than a single global threshold, is the appropriate evaluation design. The cross-bucket pooled win rate of roughly 60% to 40% is why Chapter 6 reports macro-F1 alongside accuracy and ROC-AUC as co-primary metrics.

5.4 Key Structural Insights

Five insights carry forward into the prediction chapter:

1. **The graph is hub-dominated.** A small number of cross-bucket statutes and courts (IPC, CrPC, Supreme Court) are the spine of the graph; most other entities are long-tail leaves. Any claim the model makes about “cross-case reasoning” must be interpretable against this topology.
2. **Cross-domain sharing is real but uneven.** The four IPC/CrPC-driven buckets share heavily with each other; land-property partially detaches via its own statutory backbone. A cross-domain model should outperform a naive concatenation only if it can actually exploit the shared spine.
3. **Identity-proxy leakage is structural.** Judges, specific lawyers, and specific courts have bridge and betweenness scores high enough to make *sharing* them across cases unsafe; the decision to hold them local is vindicated by the bridge analysis.
4. **Connectivity score predicts modelling difficulty.** Low-connectivity cases have, by construction, fewer shared-entity neighbours to aggregate from; the per-domain accuracy pattern in Chapter 6 is consistent with connectivity being a first-order driver of case-level difficulty.
5. **The per-domain fingerprints are legally coherent.** Each bucket’s top-PageRank entities are exactly what a lawyer would expect; this is the single strongest evidence that the pipeline’s NER, canonicalisation, and rhetorical-role filtering together produce a legally faithful, not merely statistically clean, corpus.

Chapter 6

Prediction Model and Results

6.1 Why a Heterogeneous Graph Transformer?

The exploration in Chapter 5 made two demands of the model. First, the graph has *many node types* and each carries its own semantics – a STATUTE is not a CASE, and message-passing with a shared weight matrix would force the two to live in the same geometry. Second, the graph has *many edge types* and the structural role of an edge depends on its type: `cites_statute`, `heard_in`, and `belongs_to_statute` should not share aggregation weights.

The Heterogeneous Graph Transformer (HGT) satisfies both requirements. It maintains per-node-type projections, per-edge-type messages, and multi-head attention over the heterogeneous neighbourhood, which is what a typed message-passing layer needs to reason about the graph as the relational object it is rather than as a flat homogeneous approximation. Compared to the two natural alternatives – RGCN and HAN – HGT additionally learns the relative importance of different relations via attention rather than via a fixed per-relation weight, which turns out to matter on this graph because the predictive weight of (say) `arguments` $\rightarrow$ `cites_statute` $\rightarrow$ `statute` is not the same as `case` $\rightarrow$ `heard_in` $\rightarrow$ `court`.

6.2 Model Architecture

Input projection. Every node type is first mapped to a common hidden dimension by a per-type linear layer (`input_projections`), and a learnable type embedding (`type_embeddings`, initialised at $\mathcal{N}(0, 0.02^2)$) is added. This

preserves the heterogeneity of the input features (1024-dim `bge-m3` for text nodes, concatenated scalar features for entity nodes) while giving the message-passing layers a uniform geometry to work in.

Stacked HGT layers. Two HGTConv layers with four attention heads and hidden dimension 128 form the message-passing core. Each layer is wrapped in a residual connection with per-node-type `LayerNorm`, dropout ($p = 0.2$), and a ReLU non-linearity. Two layers is deliberate: the graph diameter between a CASE and a shared STATUTE is two hops (case $\rightarrow$ arguments $\rightarrow$ statute), so two layers is sufficient to move information between the two kinds of signal the model needs – the case’s own textual evidence and the cross-case statutory neighbourhood.

Classifier head. The CASE node’s final hidden representation is fed through a two-layer `MLPHead` producing the binary outcome logits.

Training configuration. AdamW with learning rate 1×10^{-3} and weight decay 1×10^{-5} , cross-entropy loss with balanced class weights, 60 epochs with best-validation-F1 early stopping (patience 10). The loss and optimiser run on the full transductive graph; only the case-node indices are masked into train/val/test partitions. The `party_args_lr_decay` variant additionally applies a layer-wise learning-rate decay to the party-and-argument sub-module, which, as Section 6.6 shows, is the best-performing configuration.

6.3 Experimental Setup

Transductive protocol. Training is *transductive*: the full graph – including the test cases’ text nodes and entity neighbourhoods – is visible to the model, but the test cases’ *labels* are withheld through a case-node mask. This matches the real-world use case (a new judgment arrives and its graph is built using the same canonical entities as the training cases) and it is what allows shared hub nodes like IPC to carry information across the split.

5-fold cross-validation. Cases are partitioned into 5 stratified folds (seeds 42–46). For each fold: 72% train, 8% validation, 20% test. The validation fold drives early stopping; the test fold is evaluated once per fold with the best

validation checkpoint. Reported numbers are the mean and sample standard deviation across the 5 folds.

Evaluation settings. Six evaluation settings are produced per experiment: one *cross-bucket* setting that pools all five legal domains into a single 72k-case dataset, and five *per-bucket* settings that restrict the graph and the labels to a single domain. The per-bucket runs share the same architecture and training configuration; only the graph and the cases differ.

Metrics. Primary: macro-F1 (robust to the mild outcome imbalance). Secondary: accuracy, ROC-AUC. Per-class precision, recall, F1, and support are reported in the supplementary JSON logs but omitted from the text for brevity.

6.4 Ablation Variants and Rationale

Seven ablations, each a targeted structural or featural removal from the baseline:

- **case_node_minimised** – strip the CASE node of everything except the minimum required for classification, so that all case-specific information has to flow through text and entity neighbours.
- **no_names** – remove party, lawyer, and judge name content from node features; keeps structural connectivity.
- **no_cross_case** – disable sharing of STATUTE, PROVISION, and PRECEDENT across cases; every case is an isolated star.
- **text_only** – remove entity nodes entirely; the graph becomes a case-plus-text-only hierarchy.
- **hierarchical_enc** – enable hierarchical chunked encoding for all text nodes (long arguments are mean-pooled across overlapping chunks rather than hard-truncated).
- **section_sep_enc** – encode each rhetorical-role section with a distinct encoder head so their features live in disjoint subspaces.

- `party_args_lr_decay` – apply layer-wise learning rate decay to the party-and-argument sub-module so those sub-streams learn slower and remain more faithful to their pre-trained initialisations.

6.5 Baseline Comparisons

Before comparing the graph ablations against one another, it is useful to anchor the task with the strongest non-graph reference models and the unmodified HGT baseline.

Configuration	Setting	Accuracy	Macro-F1
Majority-class baseline	Fold-0 reference	0.609	0.378
Raw-feature logistic regression	Fold-0 reference	0.744	0.736
HGT baseline	5-fold cross-bucket mean	0.7811 ± 0.0055	0.7759 ± 0.0043

Table 6.1: Reference baselines for the pooled cross-bucket task. The first two rows come from the fold-0 probing package and are included to anchor task difficulty; the HGT row is the full 5-fold cross-bucket baseline used throughout the main result tables.

This baseline view clarifies the scale of the modelling problem before the ablation tables are introduced. A plain majority predictor is inadequate, and even a strong non-graph reference built on the raw pre-GNN case features reaches only 0.744 accuracy. The full HGT baseline improves that by roughly three-and-a-half points in accuracy and four points in macro-F1, which is enough to justify treating the heterogeneous graph as more than a decorative representation.

6.6 Cross-Bucket Results

Pooling the five legal domains into a single transductive graph yields the cross-bucket evaluation reported in this section ($N=72,735$ cases, 5-fold CV). The headline measurement is macro-F1 of every variant on every per-bucket dataset and on the pooled cross-bucket task; Table 6.2 collects these numbers and forms the master view of the experiments. The ablation deltas, the per-class behaviour, and the supplementary accuracy and ROC-AUC tables that follow all decompose this single table from different angles.

Model	Family	Fin. Fraud	Land/Prop.	Motor Acc.	Sexual Off.	Cross-bucket	Mean ₅
LR (no graph) [†]	0.7583	0.6961	0.7184	0.7622	0.6977	0.7372	0.7265
text_only	0.8025 ± 0.008	0.7535 ± 0.006	0.7883 ± 0.011	0.7954 ± 0.005	0.7544 ± 0.007	0.7671 ± 0.005	0.7788
baseline (HGT)	0.8042 ± 0.009	0.7577 ± 0.010	0.7916 ± 0.006	0.8147 ± 0.005	0.7538 ± 0.007	0.7759 ± 0.004	0.7844
hierarchical_enc	0.8073 ± 0.009	0.7570 ± 0.011	0.7908 ± 0.010	0.8157 ± 0.011	0.7557 ± 0.005	0.7754 ± 0.003	0.7853
section_sep_enc	0.8077 ± 0.010	0.7631 ± 0.014	0.7947 ± 0.009	0.8158 ± 0.003	0.7540 ± 0.007	0.7831 ± 0.009	0.7871
party_args_lr_decay	0.8056 ± 0.004	0.7579 ± 0.011	<u>0.7951 ± 0.005</u>	0.8164 ± 0.006	0.7596 ± 0.004	0.7959 ± 0.003	0.7869
no_names	0.8066 ± 0.006	<u>0.7599 ± 0.011</u>	0.7957 ± 0.012	0.8192 ± 0.005	0.7537 ± 0.008	0.7797 ± 0.007	<u>0.7870</u>
no_cross_case	0.8058 ± 0.009	0.7580 ± 0.013	0.7941 ± 0.008	0.8146 ± 0.006	0.7521 ± 0.008	0.7767 ± 0.003	0.7849
case_node_minimised	0.8006 ± 0.005	0.7575 ± 0.010	0.7862 ± 0.006	<u>0.8192 ± 0.010</u>	0.7514 ± 0.006	0.7697 ± 0.002	0.7830

Table 6.2: Macro-F1 across all variants and domains, 5-fold mean $\pm$ sample standard deviation. **Bold** marks the best value per column; underline marks the second-best. **Mean₅** is the unweighted average across the five per-domain columns and excludes the cross-bucket column. [†]LR (no graph) is logistic regression on the BGE-M3 case-text embedding plus the 12 scalar case features (no graph, no message passing); the LR row is reported on fold 0 from the probing pipeline and is included to anchor the “what does the graph buy us” delta against the HGT rows.

Four observations anchor the rest of the chapter.

Graph beats no-graph by a clear margin. The gap between the LR (no graph) reference and the HGT baseline is the honest measurement of what the graph contributes. On the pooled cross-bucket task it is roughly +3.9 points of macro-F1 (0.7372 $\rightarrow$ 0.7759), and against the best HGT variant (*party_args_lr_decay*) it widens to roughly +5.9 points (0.7372 $\rightarrow$ 0.7959). The same pattern holds per-bucket: every HGT variant outperforms LR by between +5 and +10 points of macro-F1, with the largest gaps on *Motor Accidents* and on the two adversarial buckets, *Sexual Offences* and *Financial Fraud*. The graph is not cosmetic.

The pooled winner is clear, the per-bucket optimum is not. On the pooled cross-bucket task, *party_args_lr_decay* is the strongest configuration at 0.7959 macro-F1, a non-trivial +2.0-point gain over the plain HGT baseline. On the per-bucket columns, however, no single variant sweeps every domain: *section_sep_enc* leads on Family and Financial Fraud, *no_names* leads on Land/Property and Motor Accidents, and *party_args_lr_decay* leads on Sexual Offences and on the pooled task. Many per-bucket margins are inside ± 0.005 , so on individual domains several variants should be read as a cluster rather than a strict ranking; the *Mean₅* column makes this explicit by collapsing the five per-domain rankings into one number, where

`section_sep_enc` (0.7871), `no_names` (0.7870), and `party_args_lr_decay` (0.7869) are statistically indistinguishable.

Identity removal does not hurt. `no_names`, which strips party, lawyer, and judge *names* from the node features, scores at or above the baseline on every bucket, ties for the best per-bucket macro-F1 on Land/Property and Motor Accidents, and is second-best on Mean₅. This is the operational answer to the identity-proxy leakage concern raised in Section 4.3: removing identity content neither helps nor hurts the model on average, which is the signature of a system that is not using identity as a shortcut.

Cross-case sharing helps modestly in the transductive regime. `no_cross_case`, which disables sharing of STATUTE, PROVISION, and PRECEDENT across cases, scores within noise of the baseline on every bucket. That does not mean cross-case edges are useless, the hub structure characterised in Chapter 5 still shows they carry genuine signal, but it does mean that in the transductive pooled regime, in-bucket connectivity already provides the model with enough cross-case structure that explicitly removing shared authority hubs is largely compensated by what remains. This is one of the more important findings of the ablation and is discussed further in Section 6.8.

6.7 Per-Domain Results

The per-domain difficulty profile of the HGT baseline is unchanged from the original transductive setup and is reproduced in Table 6.3 for reference. It serves as the single-domain anchor that the cross-bucket ablations in Table 6.2 should be read against.

Bucket	Accuracy	Macro-F1	ROC-AUC
Family & Matrimonial	0.8217 ± 0.0079	0.8042 ± 0.0088	0.8923
Financial Fraud	0.7683 ± 0.0093	0.7577 ± 0.0103	0.8406
Land & Property	0.8083 ± 0.0050	0.7916 ± 0.0060	0.8645
Motor Accidents	0.8420 ± 0.0043	0.8147 ± 0.0052	0.9010
Sexual Offences	0.7752 ± 0.0071	0.7538 ± 0.0067	0.8442
Cross-bucket pooled	0.7811 ± 0.0055	0.7759 ± 0.0043	0.8674

Table 6.3: Per-domain HGT baseline 5-fold CV results. Motor accidents is the easiest domain; sexual offences the hardest.

The ranking of domains by difficulty is stable across every ablation configuration in Table 6.2: *Motor Accidents* > *Family & Matrimonial* $\approx$ *Land & Property* > *Financial Fraud* $\approx$ *Sexual Offences*. This ordering tracks two structural properties identified in Chapter 5: (i) motor-accidents cases concentrate on a very small set of provisions and courts, producing highly repetitive, high-connectivity sub-graphs that the GNN generalises over easily; (ii) sexual-offences and financial-fraud carry the largest outcome ambiguity at the summary level (hearings frequently remand or partially dispose, which complicates the binary-outcome target produced by the Mistral labeller) and the smallest per-case sentence-level structural signal.

The cross-bucket pooled dataset is *harder* than the average of the per-domain baseline datasets (0.7759 pooled macro-F1 vs. 0.7844 Mean₅): pooling exposes the model to domain-specific label noise and cross-domain distribution shift that each single-domain model can avoid. The `party_args_lr_decay` variant closes most of this gap on the pooled task while remaining within the per-bucket cluster.

6.8 Ablation Findings

To make the contribution of each ablation legible at a glance, Table 6.4 reports macro-F1 *deltas* relative to the HGT baseline, computed cell-by-cell from Table 6.2. Positive entries are improvements over the baseline; negative entries are regressions.

Variant	Family	Fin. Fraud	Land/Prop.	Motor Acc.	Sexual Off.	Cross-bucket
<code>text_only</code>	−0.0017	−0.0042	−0.0033	−0.0193	+0.0006	−0.0088
<code>hierarchical_enc</code>	+0.0031	−0.0007	−0.0008	+0.0010	+0.0019	−0.0005
<code>section_sep_enc</code>	+0.0035	+0.0054	+0.0031	+0.0011	+0.0002	+0.0072
<code>party_args_lr_decay</code>	+0.0014	+0.0002	+0.0035	+0.0017	+0.0058	+0.0200
<code>no_names</code>	+0.0024	+0.0022	+0.0041	+0.0045	−0.0001	+0.0038
<code>no_cross_case</code>	+0.0016	+0.0003	+0.0025	−0.0001	−0.0017	+0.0008
<code>case_node_minimised</code>	−0.0036	−0.0002	−0.0054	+0.0045	−0.0024	−0.0062

Table 6.4: Macro-F1 change of each ablation relative to the HGT baseline. Each entry is the corresponding cell of Table 6.2 minus the **baseline** (HGT) row.

The findings:

1. **Removing the entity layer is the single largest drop.** `text_only`, which keeps the case-and-text sub-graph but removes statutes, provisions, precedents, and identity entities, loses −1.9 points of macro-F1 on Motor Accidents and −0.9 points on the pooled task. The entity layer therefore measurably contributes on top of the case-section graph; it is not just decorative.
2. **The party-and-argument LR-decay schedule yields the largest pooled gain.** `party_args_lr_decay` adds +2.0 points of macro-F1 on the pooled cross-bucket task, by far the largest delta in Table 6.4 and the only configuration that comfortably exceeds 0.79 pooled macro-F1. The per-bucket gains are smaller and more uniformly distributed, which suggests the variant primarily helps on the heterogeneous pooled distribution rather than on any single domain.
3. **Identity removal is at-or-above baseline on every bucket.** `no_names` is non-negative everywhere except on Sexual Offences (where it is within 0.0001 of the baseline) and is the best HGT variant on Land/Property and Motor Accidents. This is direct evidence against name leakage: party, lawyer, and judge nodes can be removed without harming the pooled or per-bucket macro-F1.
4. **Cross-case sharing is at-noise level under transductive training.** `no_cross_case` is within ± 0.003 of the baseline on every bucket, including the pooled task. The hub structure documented in Chapter 5 still carries signal, but in the transductive regime in-bucket connectivity already gives the model enough cross-case context that explicitly

disabling shared authority hubs is largely absorbed by the remaining structure.

5. **Encoder organisation matters per domain, not pooled.** `section_sep_enc` and `hierarchical_enc` both improve over baseline on Family and (for `section_sep_enc`) Financial Fraud, where the ARGUMENTS block is long, but their pooled gains are modest and they fall short of `party_args_lr_decay` on the pooled task once class balance is taken into account.
6. **case_node_minimised is a uniformly weak configuration.** Stripping the CASE node down to its 12 scalar features and forcing the GNN to reconstruct the case representation from sections and entities alone costs -0.6 points on the pooled task and degrades on every bucket except Motor Accidents. The case node’s text embedding is therefore pulling weight that the message-passing layers cannot fully recover.

6.9 Per-Class Behaviour on the Pooled Set

Macro-F1 averages across the two outcome classes. To see which class each variant actually improves, and which one is the binding constraint, Table 6.5 reports per-class precision, recall, and F1 on the pooled cross-bucket test set. Class -1 denotes *appellant lost*; class $+1$ denotes *appellant won*.

Model	Class -1 (appellant lost)			Class $+1$ (appellant won)		
	P	R	F1	P	R	F1
<code>text_only</code>	0.685	0.782	0.730	0.846	0.768	0.805
<code>baseline</code> (HGT)	0.690	<u>0.805</u>	0.742	0.860	0.765	0.810
<code>hierarchical_enc</code>	0.692	0.795	0.740	0.854	<u>0.772</u>	0.811
<code>section_sep_enc</code>	0.705	0.796	0.747	0.857	0.785	0.819
<code>party_args_lr_decay</code>	0.723	0.803	0.761	0.864	0.801	0.831
<code>no_names</code>	0.707	0.779	0.741	0.848	0.792	0.819
<code>no_cross_case</code>	0.704	0.777	0.738	0.846	0.788	0.816
<code>case_node_minimised</code>	0.686	0.788	0.733	0.849	0.767	0.806

Table 6.5: Per-class precision, recall, and F1 on the pooled cross-bucket test set. Class supports per fold are $|-1| \approx 5,625$ and $|+1| \approx 8,737$. **Bold** marks the best value per column; underline marks the second-best.

The minority outcome class -1 is consistently the harder of the two: across every variant, its precision is the lowest entry in the row.

`party_args_lr_decay` is the only configuration that lifts *both* classes simultaneously, a +3.3-point jump in $P(-1)$ over the baseline together with a +3.6-point jump in $R(+1)$, which is what produces its +2.0-point pooled macro-F1 gain. The other variants tend to trade one class against the other (`section_sep_enc`, for example, gains +1.5 pts on $P(-1)$ but loses ground elsewhere), and the per-class table makes it clear that the pooled improvement of `party_args_lr_decay` is structural rather than the artifact of a favourable class shift.

6.10 Best Model: Per-Bucket Per-Class Break-down

Table 6.6 reports the per-bucket and per-class precision, recall, F1, and support for the best pooled model, `party_args_lr_decay`. This is the finest-grained view of where the chosen model wins and where it does not.

Bucket	Class -1 (lost)				Class +1 (won)			
	P	R	F1	Sup.	P	R	F1	Sup.
Family & Matrimonial	0.731	0.758	0.744	988	0.876	0.859	0.868	1,966
Financial Fraud	0.724	0.682	0.702	1,120	0.799	0.829	0.814	1,711
Land & Property	0.823	0.881	0.851	1,630	0.790	0.698	0.740	1,027
Motor Accidents	0.700	0.806	0.749	885	0.914	0.856	0.884	2,128
Sexual Offences	0.676	0.704	0.689	1,002	0.841	0.821	0.831	1,905
Cross-bucket pooled	0.723	0.803	0.761	5,625	0.864	0.801	0.831	8,737

Table 6.6: Per-bucket per-class breakdown of the best pooled model (`party_args_lr_decay`). Reported per fold; supports are rounded to the fold-0 representative.

Three observations are worth making explicit. First, the difficulty ordering by macro-F1 (*Motor Accidents* $\approx 0.816 >$ *Family / Land & Property* $>$ *Sexual Offences* $\approx 0.760 \approx$ *Financial Fraud* ≈ 0.758) is the same ordering identified at baseline level in Table 6.3, which confirms that the best variant does not simply rearrange where the model is strong. Second, Land & Property is the only bucket in which the imbalance flips, class -1 is the majority, and the model’s -1 F1 of 0.851 is the highest single per-class entry in the table; the model is strongest where its conservative-by-default behaviour is aligned with the class prior. Third, Sexual Offences carries the lowest minority-class F1

(0.689), consistent with the noisier outcome labels in that bucket and with the broader observation in Section 6.9 that the minority class is the binding constraint everywhere.

6.11 Out-of-Domain Evaluation: A Fully Unseen Legal Domain

Problem. Every result reported so far is drawn from the same five legal domains the model was trained on, including the pooled cross-bucket evaluation, which still mixes familiar per-bucket label distributions and familiar statutory hubs. That tests *within-distribution* generalisation. A stricter question is whether the model’s learned reasoning structure transfers to a *legal domain it has never seen*: unfamiliar statutes, unfamiliar provisions, an unfamiliar argument vocabulary, and a different outcome prior. If the cross-bucket accuracy reported above were driven by memorisation of domain-specific hubs, performance on an unseen domain would collapse to the majority-class baseline; anything materially above that baseline is evidence that the model encodes domain-general legal-reasoning signal.

Approach. We run an end-to-end, zero-fine-tuning evaluation on *food-safety* judgments, a body of Indian case law governed primarily by the Food Safety and Standards Act 2006, the Prevention of Food Adulteration Act 1954, and their state-level counterparts, none of which are represented in any of the five training buckets. The protocol is:

1. **Corpus assembly.** 11,769 food-safety judgments are pulled from the same Indian-Kanoon ingest pipeline used for the main corpus and run through the *identical* preprocessing, NER, and outcome-labelling stages. No food-safety example is available to the model at training time; the only shared machinery is the pipeline itself.
2. **Graph construction.** The food-safety graph is built with the same reasoning-focused schema from Chapter 4, using the same `bge-m3` encoder for text nodes, and cached in the same format as the main per-bucket graphs. No retraining or fine-tuning is performed.
3. **Inference only.** Each of the five trained `party_args_lr_decay` fold checkpoints (the best cross-bucket configuration of Table 6.2) is loaded

unchanged and evaluated once on the full food-safety graph. Reported numbers are the mean and sample standard deviation across the five checkpoints.

Because every transformation from raw text to graph feature is identical to the training pipeline, what is being measured is the learned representation’s ability to absorb a new domain’s text, statutes, and argument structure without any parameter update.

Setting	N	Accuracy	Macro-F1	ROC-AUC
Majority-class baseline (food-safety)	11,769	0.543	–	–
Cross-bucket train $\rightarrow$ food-safety	11,769	0.6079 ± 0.0054	0.6034 ± 0.0070	0.6749 ± 0.0198
<i>In-domain reference: cross-bucket 5-fold CV</i>	72,735	0.8020 ± 0.0036	0.7959 ± 0.0028	0.8831

Table 6.7: Out-of-domain evaluation on food-safety judgments, which the model has never seen during training. Each of the five `party_args_lr_decay` fold checkpoints is applied without fine-tuning to the full food-safety graph. Accuracy is +6.5 points above the majority-class baseline and ROC-AUC is 0.67; the in-domain reference row is the best configuration from Table 6.2, provided here to show the size of the generalisation gap.

Results.

What this shows. Three observations matter. First, every fold is strictly above the majority-class baseline (0.599–0.613 accuracy against a base rate of 0.543), with +6.5 accuracy points on average. Second, the ROC-AUC of 0.675 is substantially above chance (0.5) even though the model has never been exposed to food-safety statutes, provisions, or argument patterns. Third, the across-fold standard deviation is small (± 0.005 accuracy, ± 0.007 macro-F1), which rules out the possibility that the gain comes from a lucky checkpoint rather than a property of the training protocol.

At the same time, the in-domain reference row makes clear that this is still a genuine out-of-domain regime: the model loses roughly 20 accuracy points and 21 ROC-AUC points when moved from in-distribution data to food-safety. The interesting claim is therefore not “the model transfers” in some strong sense, clearly, a lot of the learned signal does not transfer, but that the transferred signal is non-trivial and statistically stable. Whatever

the model learned during cross-bucket training is *partially* domain-general: some portion of it relies on cross-domain legal-reasoning structure (argument-to-outcome mappings, petitioner/respondent dynamics, procedural markers) rather than on memorising specific statutes or provisions. The confusion matrix is consistent with this reading: precision for the *petitioner-wins* class is 0.690 (the model is conservative about predicting a win on an unfamiliar domain), and recall for *petitioner-loses* is 0.724 (it recognises loss patterns more reliably than win patterns on the unfamiliar corpus).

The operational take-away is that the cross-bucket HGT can be used as a *zero-shot base model* for unseen Indian legal domains and will perform meaningfully above chance without fine-tuning, but that closing the remaining 20-point gap almost certainly requires exposing the model to the target domain’s statutes and argument patterns. That is a practical roadmap: re-train on the target corpus, or fine-tune the final layers on a small labelled subset; the present experiment establishes that a strong prior is already in place before either step begins.

This closes the quantitative story of the prediction model. The remaining open questions, *what* the learned representation actually encodes, *which* graph components drive the decisions, *where* the model breaks, and *why* a specific prediction is what it is, are the subject of Chapter 7, which treats the trained HGT as a frozen object and dissects it at four progressively finer levels of granularity: global representation, global dependence on graph components, failure structure, and per-case explanation.

Chapter 7

Interpretability and Case-Level Explanation

7.1 Why Interpretability, and Why at Three Levels

Chapter 6 establishes that the heterogeneous graph transformer predicts case outcomes more accurately than non-graph references and that it does so without relying on identity shortcuts. Those are necessary conditions for a legal-prediction model, but they are not sufficient. A legal audience, practitioners, auditors, reviewers, does not evaluate a prediction system purely on aggregate accuracy; it evaluates it on whether individual predictions can be *read*, and on whether the aggregate accuracy is explained by legally coherent structure rather than by incidental corpus regularities.

This chapter treats the trained HGT as a *frozen* object and dissects it through three progressively finer lenses. Presenting the analysis as a single pipeline rather than a set of unrelated tools is deliberate: each lens answers a different question but operates on the same artefacts, the pre- and post-GNN case embeddings, the frozen graph, and the trained checkpoint, so the results are directly comparable.

1. **Representation-level analysis.** *Does the GNN learn anything beyond the input features?* A probing classifier and t-SNE visualisation characterise the learned case embedding and measure the gain contributed by message passing.

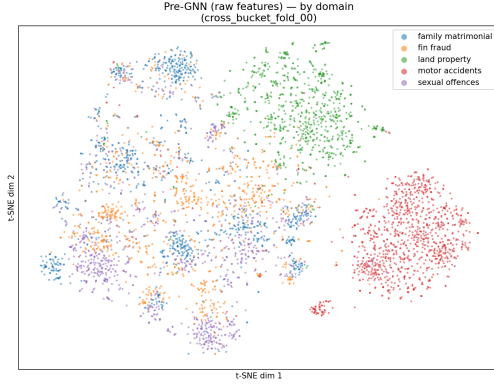
2. **Component-level analysis.** *Which node and edge types does the model actually depend on?* Node-type zeroing, per-relation attention, and SHAP over the post-GNN embedding identify the graph components whose removal most degrades performance.
3. **Case-level explanation.** *For one prediction, why?* A heterogeneous edge-masking explainer combined with LLM verbalisation produces, for a chosen test case, the statutes, provisions, precedents, and argument spans the frozen model used, and verbalises that evidence bundle into a legal narrative.

The first two levels operate globally, one model, many cases. The third operates locally, one case at a time. The bridge between them is the post-GNN case embedding: representation-level analysis establishes that this embedding carries outcome-relevant structure, component-level analysis identifies which graph signals shape it, and case-level explanation uses the trained edge masker to ground individual explanations in legally named evidence.

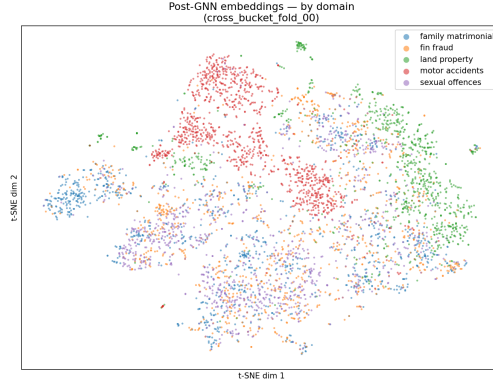
7.2 Representation-Level Analysis: What Did the GNN Learn?

The probing-classifier comparison in Chapter 6 already established that the 64-d post-GNN CASE embedding outperforms the 1036-d pre-GNN feature vector by a non-trivial margin under an identical L2 logistic-regression head and an identical train/test split. That number isolates the contribution of message passing from the underlying `bge-m3` text features, but it does not say *how* the representation has been reorganised. To see the geometry, pre- and post-GNN embeddings are projected into 2-D with t-SNE, coloured once by legal-domain bucket and once by outcome label.

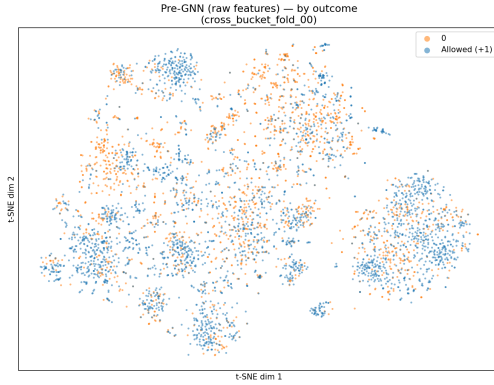
(a) Pre-GNN, coloured by domain



(b) Post-GNN, coloured by domain



(c) Pre-GNN, coloured by outcome label



(d) Post-GNN, coloured by outcome label

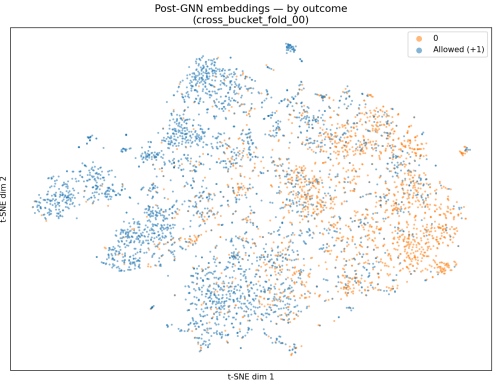


Figure 7.1: t-SNE projections of pre- and post-GNN case embeddings on fold 0 of the cross-bucket test split. Domain structure is already present in the pre-GNN view; label structure emerges only after graph propagation.

The picture is consistent with the probing numbers. Pre-GNN embeddings cluster strongly by *domain*, the **bge-m3** representation of the concatenated case text already separates the five legal buckets, but outcome labels are mixed within each cluster. Post-GNN embeddings preserve the domain clusters but reorganise each cluster internally so that outcome colour becomes visibly structured. The HGT’s contribution is therefore primarily *within-domain* label geometry, not between-domain reorganisation: message passing uses the shared hubs to sharpen outcome separation inside each legal neighbourhood.

7.3 Component-Level Analysis: Which Parts of the Graph Matter?

Problem. The representation-level analysis tells us the graph adds signal; it does not tell us *which* parts of the graph add it. The heterogeneous schema has many node types and many edge types, and the ablation table in Chapter 6 (Table 6.2) shows that removing any one of them has only a modest effect. That is a coarse test: it measures *re-trained* performance without a component. A complementary question is, for the *same* frozen model, which components is the prediction *using* on a case-by-case basis?

Three complementary probes. The component-level analysis uses three complementary probes: node-type zeroing, per-relation attention with gradient saliency, and SHAP over the post-GNN embedding. They answer different slices of the same question, and using them together reduces the risk of any single method producing a misleading ranking.

Node-type zeroing. For each node type in turn, all features of that type are replaced with zeros and the frozen HGT is re-evaluated; the drop in test accuracy is that type’s importance. Across the cross-bucket baseline the CASE node dominates and the argument-bearing text nodes together with PETITIONER and JUDGE follow.

Per-relation attention and gradient saliency. A second probe extracts per-relation attention priors and node-type gradient saliency, and broadly agrees with the zeroing analysis, saliency confirms that the argument stream is the operational hinge of the model.

SHAP over the post-GNN embedding. The third probe computes mean absolute SHAP values for each dimension of the 64-d post-GNN embedding under a logistic-regression probe.

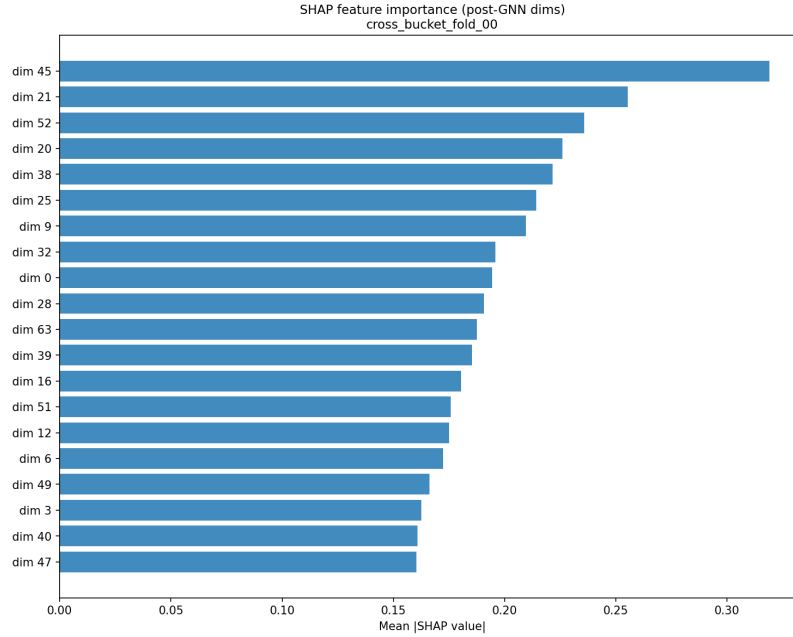


Figure 7.2: Mean absolute SHAP values per latent dimension of the post-GNN case embedding. The top six dimensions (45, 21, 52, 20, 38, 25) carry the largest marginal contribution, but no single dimension dominates.

No single dimension dominates: the top six dimensions each contribute meaningfully, but the remaining 58 dimensions still sum to a comparable share of the total mass. A distributed embedding is exactly what one would want from a graph-level representation, collapse onto a small number of dimensions would be evidence of representational bottleneck. It is worth stressing, however, that SHAP operates on the *latent* space: it identifies important embedding *dimensions*, not interpretable legal concepts. For that reason, this probe is used as a *representation diagnostic* rather than as the final legal explanation; the case-level pipeline of Section 7.4 is what produces legally named evidence.

The global probes above answer the *what* and *which* questions. What they cannot answer is the *why* for any individual prediction, why a particular case was decided the way the model says it was, or why a confident prediction nevertheless turned out wrong. That question requires a case-scale pipeline.

7.4 Case-Level Explanation

Problem. The representation- and component-level probes characterise the model in aggregate: what the GNN learned, and which graph parts it relies on on average. Neither speaks to a specific decision, why *this* case was predicted the way it was, or whether a confident prediction is grounded in legally coherent evidence. Closing that gap demands an additional, case-wise step on top of the global analyses: for any chosen test case, surface a ranked set of *legally named* entities, statutes, provisions, precedents, and argument spans that the frozen model actually relied on, and a natural-language synthesis a practitioner can read.

Approach. This case-wise step is implemented as a four-phase pipeline bolted on top of the frozen HGT checkpoint, summarised in Table 7.1. The bridge to the global analyses is the post-GNN embedding: Section 7.2 established that this embedding encodes outcome-relevant structure, while Section 7.3 identifies which graph components shape that structure.

Phase	Role
1. Inference	Run the frozen HGT once over the full graph; cache 64-d post-GNN embeddings, predicted labels, and class probabilities for every case.
2. Edge masker	Train a heterogeneous edge masker against the frozen HGT, scoring how much each edge contributes to the prediction.
3. Evidence	Convert edge scores into ranked statutes, provisions, precedents, and argument spans for the target case.
4. Verbalisation	Pass the evidence bundle to a Mistral-Small 24B model, which writes a structured legal narrative around it.

Table 7.1: Four phases of the case-level explainer. Phases 1–2 are one-time set-up; phases 3–4 are run per target case.

The experiments in this section use the *cross-bucket* pooled graph, fold 00 of the k-fold split (71,813 cases total: 51,705 training, 5,745 validation, 14,363 test), with the best-performing checkpoint from Chapter 6, namely the `party_args_lr_decay` HGT (hidden dimension 64, two message-passing layers, four attention heads, dropout 0.25). The pooled checkpoint is chosen so that evidence extraction can cross domain boundaries through the shared hub structure when the graph representation justifies it.

7.4.1 Phase 1: Frozen Inference

Phase 1 runs the frozen HGT in inference mode over the full graph and caches, for every case node, its 64-d post-GNN embedding, predicted outcome label, and per-class probabilities. These cached outputs provide the prediction context used by the later explanation steps without re-training the model.

7.4.2 Phase 2: Training a Heterogeneous Edge Masker

The textbook tool for edge attribution on GNNs is PGExplainer [10]: a small MLP scores each edge, and the top-scoring subset, fed back through the model, must preserve the prediction. PyG’s implementation is *homogeneous*, it weights messages through `edge_weight`, which HGTConv does not accept, since its attention is defined over typed neighbourhoods rather than scalar-weighted messages.

Heterogeneous adaptation. The explainer keeps the PGExplainer objective but changes the mask semantics from *weighting* edges to *selecting* them. A shared MLP, fed source- and destination-node embeddings together with a learnable relation-type embedding, emits a binary mask; Phase 2 drops the masked edges from the typed edge dictionary and reruns the frozen HGT on the reduced graph. The mask is relaxed to $[0, 1]$ via Gumbel-sigmoid straight-through estimation, keeping selection discrete at inference time while remaining differentiable during training. The objective combines three terms:

1. **Prediction loss:** cross-entropy between the frozen HGT’s predictions on the masked and full graphs, drives the mask to preserve the model’s output.
2. **Size regulariser:** an ℓ_1 penalty on the fraction of selected edges, encourages sparse explanations.
3. **Entropy regulariser:** an entropy penalty on the edge-score distribution, prevents collapse to uniform weights and forces hard decisions.

Temperature is annealed from 5 to 1 over 30 epochs: soft exploration early, hard selection at convergence.

7.4.3 Phase 3: From Edge Scores to Legal Evidence

An edge-level score is not yet a legal explanation. Phase 3 translates the trained masker’s output into entity-level answers. Edge importance is computed once globally rather than per case, so each new explanation only has to read scores off a precomputed table. For each target case:

1. Collect the k -hop subgraph centred on the case by undirected BFS, with depth matching the HGT’s two message-passing layers, exactly the nodes whose information the frozen model could have propagated into the case representation.
2. Restrict the precomputed edge-importance table to edges incident to that reachable set; the frozen HGT is not re-run.
3. Aggregate edge scores to node importance by the max-incident-edge rule, and rank nodes by type to produce top- N lists for STATUTE, PROVISION, PRECEDENT, and the four argument-node variants.

The output is a JSON record containing predicted and true labels, confidence scores, ranked entity lists, and the raw numerical evidence backing each claim.

7.4.4 Phase 4: LLM Translation

A ranked list of node identifiers is machine-readable but not human-readable: a legal explanation needs the statutes tied to the provisions, the provisions tied to the case facts, and the case facts tied to the argument evidence. Phase 4 passes the Phase 3 record to a Mistral-Small 24B model run locally under vLLM, prompted to write a structured narrative covering the predicted outcome, governing statutes, key provisions, salient precedents, argument patterns, and a plain-language reasoning summary.

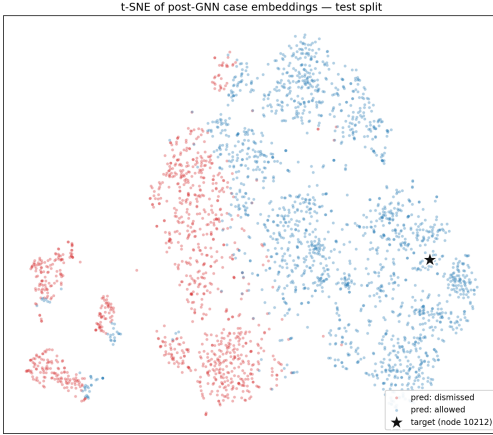
Role boundary. The LLM is not an open-ended legal reasoner: its inputs are the evidence bundle from Phase 3 plus the pre-outcome text of the target case (preamble and arguments). The true outcome is supplied as an “*actual outcome (for reference)*” field so that the narrative can flag model errors instead of silently rationalising them, but the prompt explicitly asks the LLM to explain the *predicted* label, not to re-decide the case. Evidence selection

stays with the trained explainer; verbalisation stays with the LLM. That separation is what makes the resulting narrative attributable to the HGT rather than to LLM priors.

7.5 A Worked Example: A Confidently-Correct Case

To illustrate the pipeline end-to-end, consider test case 10212 (*Sanjeev Kumar vs State of Punjab and Another*, 19 April 2023). The frozen HGT predicts *petitioner wins* with probability 0.9998; the ground-truth label is also *petitioner wins*. This is the kind of prediction where a global accuracy number gives no information about whether the model got the case right for the right reasons.

(a) Target case in post-GNN t-SNE space



(b) Top evidence subgraph from the PGExplainer

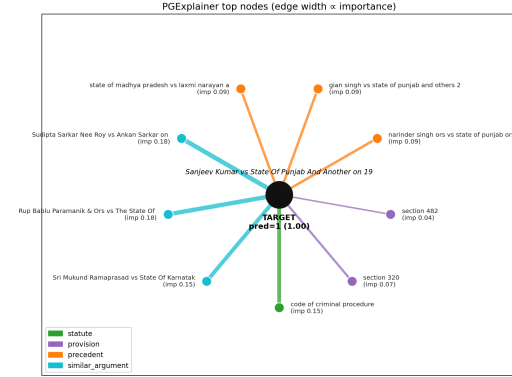


Figure 7.3: Confidently-correct case 10212 (*Sanjeev Kumar vs State of Punjab and Another*, 2023). Panel (a) locates the case in post-GNN embedding space; panel (b) shows the top-weighted statutes, provisions, precedents, and argument nodes selected by the trained edge masker.

Evidence extracted. The governing statute surfaced by the masker is the *Code of Criminal Procedure*. The top two provisions are Section 320 CrPC (compounding of offences) and Section 482 CrPC (inherent power of the High Court to quash proceedings), exactly the doctrinal pair that underlies the family-matrimonial “quash-on-compromise” pattern.

LLM-translated narrative. The Phase 4 narrative reconstructs this as standard quash-on-compromise reasoning: the parties have settled the dispute; Section 320 CrPC governs compounding; Section 482 CrPC authorises the High Court to quash proceedings to prevent abuse of process; *Gian Singh vs State of Punjab* (2012) and *State of MP vs Laxmi Narayan* (2019) establish that inherent-power quashing is available even for non-compoundable offences when the compromise is genuine. This is not the LLM inventing a legal argument: every named statute, provision, and precedent in the narrative comes from the Phase 3 evidence bundle. What the LLM supplies is the connective tissue between those pieces.

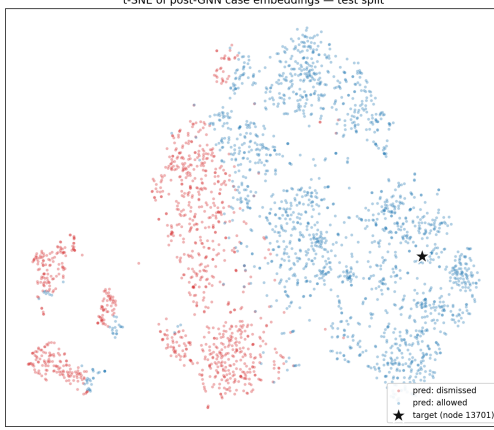
7.6 When Confidence and Correctness Diverge: Structural Error Analysis

The interesting failures of a prediction model are not the low-confidence mistakes, those are honest and already reflected in the calibration, but the *high-confidence* mistakes. The case-level explainer in Section 7.4 provides a natural diagnostic tool for them: inspect whether the selected statutes, provisions, precedents, and argument nodes explain a legally plausible but ultimately incomplete path to the predicted label.

Interpretation. Confident errors are not random. They occur when the high-scoring evidence subgraph preserves enough legally relevant structure to make the wrong label look coherent, while case-specific distinguishing facts remain under-represented. This is a sharper failure mode than “the model is uncertain”. It means that the same mechanism used to explain successes can also expose how a confident mistake was made.

A worked confident-wrong example. Test case 13701 (*Umesh V vs State of Kerala*, 18 January 2023) is predicted *petitioner wins* with probability 0.9998; the true label is *petitioner loses*. The explanation is useful precisely because it shows which graph-local evidence supported the wrong prediction rather than merely reporting that the classifier was confident.

(a) Target case in post-GNN t-SNE space



(b) Top evidence subgraph

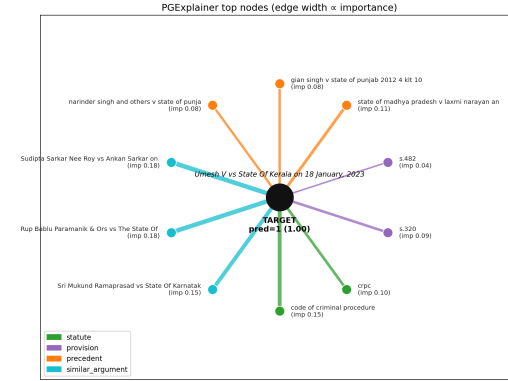


Figure 7.4: Confidently-wrong case 13701 (*Umesh V vs State of Kerala*, 2023). The target is explained through a compact evidence subgraph whose selected structure supports the predicted outcome even though the actual outcome is loss.

The evidence subgraph is not *wrong* in any local sense: it surfaces legally relevant structure that the model can use. What the failure reveals is that a small number of distinguishing facts, the specific factual configuration that led this particular court to refuse relief, do not survive strongly enough in the explanation.

7.7 Summary

The interpretability pipeline presented in this chapter provides a coherent story for the trained HGT. At the *representation* level, a pre-GNN/post-GNN probe shows a +3.97-point accuracy gain attributable to message passing, and t-SNE confirms that the gain takes the form of within-domain outcome reorganisation. At the *component* level, node-type zeroing, per-relation attention, and SHAP all converge on the same story: the CASE node dominates, but argument nodes and the immediate participants contribute measurable graph-local signal that a plain text classifier cannot access. At the *case* level, frozen inference, heterogeneous PGExplainer, and LLM translation produce grounded, legally named explanations for individual predictions.

Crucially, the same pipeline exposes the model’s *confident* failure mode. For high-confidence predictions that are nevertheless wrong, the selected

evidence can still form a legally plausible but incomplete explanation for the predicted label. Making that visible is what distinguishes a post-hoc explainer from a calibration metric, and it is the part of the pipeline with the clearest operational value for deployment.

Chapter 8

Discussion

8.1 Summary of Findings

The thesis pursued a single operational question: can a heterogeneous knowledge graph built from Indian court judgments, filtered to be structurally leakage-safe, support an outcome-prediction model whose accuracy is attributable to legal reasoning rather than to identity or decision shortcuts? The chapters above answer that affirmatively but with important qualifications.

The pipeline produces a legally coherent corpus: the top-PageRank entities recovered per bucket are exactly the statutes, provisions, and courts a practitioner would expect, and the leakage-focused pruning of Chapter 4 (dropping auxiliary NER types, forbidding shortcut edges from identity proxies, keeping courts and counsel local) is restrictive without being destructive: the resulting graph is large, connected, and hub-dominated, with IPC, CrPC, and the apex courts bridging all five legal domains.

The HGT learns cross-case structural signal that is not reducible to its text features. The post-GNN probing classifier outperforms the pre-GNN baseline at fixed capacity with a $16\times$ smaller representation, and the t-SNE geometry shows that message passing reorganises an already-strong text representation into a more outcome-informative case embedding rather than replacing text with a pure hub-traversal signal. The graph’s contribution is best described as structured aggregation of legal context, with cross-case shared authorities providing a secondary, domain-dependent benefit.

The model resists identity shortcuts at four independent levels. At the

schema level, courts, judges, and lawyers are local and shortcut edges from identity proxies are forbidden; at the text level, rhetorical-role and phrase filtering remove direct decision leakage; at the model level, the `no_names` ablation matches the baseline on every bucket; and at the representation level, probing, saliency, and error analysis converge on the same conclusion. The remaining difficult cases are the doctrinally crowded and document-heavy judgments, not the de-identified ones.

Predictions are auditable case by case. The `Graph_Analyser` pipeline of Chapter 7 surfaces, for any test case, the statutes, provisions, precedents, and argument spans the frozen model relied on, and verbalises the bundle through a Mistral-Small narrative. The same machinery exposes a structural signature of confident error: the selected evidence can make the predicted label look legally plausible while missing distinguishing facts that explain the true outcome.

The cross-bucket representation also transfers, partially, to a fully unseen legal domain. Evaluated zero-shot on 11,769 food-safety judgments (Section 6.11) the model is strictly above the majority-class baseline with low across-fold variance, which is evidence that the trained HGT carries a partially domain-general legal-reasoning prior rather than memorising the training domains’ statutory hubs. The ~ 20 -point gap to in-domain performance is substantial and bounds the strength of the claim.

8.2 Limitations

Transductive evaluation. The main Chapter 6 numbers are produced under a transductive protocol, in which the full graph, including test cases’ entity neighbourhoods, is visible during training. This matches a realistic deployment setting (a new judgment is appended to an existing canonical vocabulary), and the food-safety experiment partially relaxes it by evaluating on an entirely unseen legal domain. What the present evaluation does not measure is finer-grained inductive generalisation within a familiar domain, a calendar-year or court hold-out, where most entities are known but the specific case neighbourhood is new. The per-year error pattern suggests older cases are harder, which is a prior on how such a split would behave.

LLM-generated outcome labels. The win/loss labels are produced by prompting Mistral on the OpenNyAI summary of each case and are therefore

noisy with respect to the underlying disposition. The noise is not uniform: sexual-offences and financial-fraud matters often have multi-part dispositions that do not map cleanly onto a binary axis, and these are also the two worst-performing buckets, at least partly a label-noise effect rather than a modelling failure.

Leakage filtering is a lower bound, not a guarantee. The deterministic filters (rhetorical-role labels, field names, leakage phrases) plus the structural argument (shared-entity types are law, not identity) are neither complete: an RR classifier can mis-label a decisive sentence, the phrase list is never exhaustive, and a sufficiently expressive GNN could in principle reconstruct a decisive fragment from non-decisive neighbours. The `no_names` result addresses one specific leakage mode, not all of them.

Small ablation margins and non-universal pooled optimum. The best configuration beats the baseline by +1.7 accuracy points on the pooled dataset, with 5-fold error bars of similar magnitude, so close ablations should be read as a cluster of comparable configurations rather than a strict ranking. The per-dataset best variant also differs by domain: family-matrimonial, motor-accidents, and sexual-offences favour different ablations under accuracy or macro-F1, so the pooled cross-bucket score is a useful compromise over heterogeneous graph geometries rather than evidence that one architectural setting is intrinsically best everywhere.

8.3 Future Work

Four directions follow most directly from the findings and limitations above.

Stronger generalisation and leakage audits. The natural next experiments are inductive splits that the present transductive protocol cannot evaluate: a calendar-year hold-out (e.g. train pre-2020, test post-2020) and a court hold-out (train on all but one High Court). The same hold-out machinery doubles as a sharper leakage probe, a judge-hold-out and a court-hold-out would tighten the identity-leakage claim from “no evidence of shortcuts” to “shortcuts are quantitatively bounded”. The connectivity-score diagnostic of Chapter 5 is the natural metric for these splits.

Richer outcome targets. The Mistral labeller already produces a multi-label variant alongside the binary `win/loss` target used here. Moving the classifier head to a multi-label sigmoid or an ordinal regression over the $[-1, +1]$ score would absorb the disposition ambiguity that currently inflates label noise on sexual-offences and financial-fraud matters and would enable disposition-specific error analysis (*remanded* versus *dismissed*).

Cross-domain transfer and domain-adaptive heads. The food-safety experiment shows the cross-bucket HGT is a usable zero-shot base on an unseen domain but also quantifies the ~ 20 -point in-domain/out-of-domain gap. Two complementary follow-ons map onto that finding: (i) light-weight fine-tuning of the classifier head (with an optional CASE-node adapter) on a modest labelled target-domain sample, to measure how quickly the gap closes, and (ii) a domain-conditioned head that reads a domain embedding alongside the post-GNN case embedding so that one unified model can be evaluated across training and held-out domains without per-domain retraining. Scaling the bucket inventory to additional domains (tax, labour, consumer) is a related engineering exercise that the pipeline already supports.

Expert-in-the-loop validation of case-level explanations. The `Graph_Analyser` pipeline already emits, for any case, the ranked statutes, provisions, precedents, and argument spans the frozen HGT relied on. Sampling predictions across confidence and difficulty strata and putting the resulting evidence bundles in front of a legal expert would test (i) whether the surfaced authorities are legally defensible reasons for the predicted outcome, (ii) whether the selected argument evidence captures the legally decisive facts, and (iii) whether confident-wrong cases correspond to a coherent class of distinguishing-fact omissions. That turns the explainer from an internal diagnostic into an auditable artefact and is the cleanest path from offline evaluation to a deployable review tool.

Taken together, the contributions of this thesis are a leakage-safe construction pipeline for legal knowledge graphs, an HGT-based prediction model whose accuracy survives an aggressive identity-leakage audit, a case-level interpretability pipeline that exposes both correct reasoning and a structural signature of confident error, and a zero-shot evaluation on a fully unseen legal domain that quantifies the cross-domain transferability of the learned representation.

Chapter 9

Conclusion

This thesis studied Legal Judgment Prediction for Indian court cases as a structured prediction problem rather than a plain document-classification task. It argued that credible legal prediction requires more than strong accuracy numbers: it requires a representation that preserves legally meaningful structure while actively resisting leakage from decision-bearing text and identity-based shortcuts.

To address this, the thesis introduced an end-to-end pipeline that converts raw Indian judgments into leakage-audited, graph-ready case records through entity extraction, rhetorical-role segmentation, outcome labelling, multi-hearing consolidation, and reasoning-focused graph construction. On top of this representation, a Heterogeneous Graph Transformer was used to model both the internal structure of each case and the shared legal links across cases through statutes, provisions, and precedents.

Across more than 73,000 Indian court cases spanning five legal domains, the proposed framework achieved strong pooled performance (0.8020 accuracy, 0.7959 macro-F1, 0.8831 ROC-AUC under 5-fold cross-validation on the best `party_args_lr_decay` configuration) and consistently outperformed text-only reference settings. More importantly, the results and ablations suggest that the gains are not coming merely from surface-level text patterns, but from cross-case legal structure encoded in a controlled and auditable way: a probing classifier isolates a +3.97-point accuracy gain attributable to graph message passing at fixed classifier capacity.

Beyond pooled accuracy, the thesis addressed the two questions a legal audience actually cares about: *can individual predictions be read*, and *does the model transfer beyond the distribution it was trained on*? A four-level

interpretability pipeline, representation probing, component-level importance, error decomposition, and the case-level **Graph_Analyser** built around a heterogeneous PGExplainer and LLM verbalisation, produces grounded, legally named explanations for individual predictions and exposes a clean structural signature of the model’s confident failures: when the prediction is confidently wrong, the selected evidence can still form a legally plausible but incomplete explanation for the *wrong* label. A zero-shot evaluation on 11,769 food-safety judgments, a legal domain absent from training, reaches 0.608 accuracy and 0.675 ROC-AUC against a 0.543 majority baseline without any fine-tuning, showing that the cross-bucket HGT learns a partially domain-general legal-reasoning prior rather than memorising training domains’ statutory hubs.

The main contribution of this thesis is therefore methodological. It shows that legal outcome prediction over Indian judgments becomes more credible when the task is built around leakage-safe data construction, heterogeneous graph design, explicit constraints on what information may be shared across cases, per-case explainability tied to the same representation the model predicts from, and an out-of-domain evaluation that tests whether the learned signal is truly about legal reasoning or about corpus memorisation. While the improvements are measured rather than dramatic, they support a clear conclusion: graph-based legal prediction is most useful when it is not only accurate, but also structurally faithful, auditable at the case level, and experimentally honest about its transfer behaviour.

Future work can build on this foundation by tightening leakage audits further, extending the outcome space beyond binary decisions, designing models that adapt to domain-specific difficulty and long factual narratives, closing the in-domain/out-of-domain gap through light-weight fine-tuning or domain-adaptive heads, and pairing the case-level explainer with a legal-expert validation loop to convert it from an internal diagnostic into an auditable review tool.

Bibliography

- [1] Nikolaos Aletras, Dimitrios Tsarapatsanis, Daniel Preşiu-Pietro, and Vasileios Lamps. Predicting judicial decisions of the European Court of Human Rights: a Natural Language Processing perspective. *PeerJ Computer Science*, 2:e93, 2016.
- [2] Paheli Bhattacharya, Kaustubh Hiware, Subham Rajgaria, Nilay Pochhi, Kripabandhu Ghosh, and Saptarshi Ghosh. A comparative study of summarization algorithms applied to legal case judgments. In *Advances in Information Retrieval – ECIR 2019*, volume 11438 of *Lecture Notes in Computer Science*, pages 413–428. Springer, 2019.
- [3] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2898–2904. Association for Computational Linguistics, 2020.
- [4] Ilias Chalkidis, Abhik Jana, Dirk Hartung, Michael Bommarito, Ion Androutsopoulos, Daniel Katz, and Nikolaos Aletras. LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4310–4330. Association for Computational Linguistics, 2022.
- [5] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, pages 2704–2710. ACM, 2020.
- [6] Abhinav Joshi, Shounak Paul, Avinash Anand, Md. Shad Akhtar, and Ashutosh Modi. IL-TUR: Benchmark for Indian legal text understanding

- and reasoning. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 4035–4049. ELRA and ICCL, 2024.
- [7] Prathamesh Kalamkar, Aman Tiwari, Astha Agarwal, Saurabh Karn, Smita Gupta, Vivek Raghavan, and Ashutosh Modi. Corpus for automatic structuring of legal documents. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 4420–4429. European Language Resources Association, 2022.
 - [8] Daniel Martin Katz, Michael James Bommarito, and Josh Blackman. A general approach for predicting the behavior of the Supreme Court of the United States. *PLOS ONE*, 12(4):e0174698, 2017.
 - [9] Shubham Khatri, Pooja Rani, Aaryan Mattoo, Bhanu Pratap Singh Rawat, Kshitij Gulati, Shouvik Sachdeva, Gaurav Gambhir, and Vikram Goyal. Legal judgment prediction with Indian court cases using graph neural networks. In *Proceedings of the 19th International Conference on Artificial Intelligence and Law*, pages 191–200. ACM, 2023.
 - [10] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.
 - [11] Vijit Malik, Rishabh Sanjay, Shubham Kumar Nigam, Kripa Ghosh, Shouvik Sengupta, Arnab Bhattacharya, and Ashutosh Modi. ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4046–4062. Association for Computational Linguistics, 2021.
 - [12] Dimitris Mamakas, Petros Tsotsi, Ion Androutsopoulos, and Ilias Chalkidis. Processing long legal documents with pre-trained transformers: Modding LegalBERT and Longformer. In *Proceedings of the Natural Legal Language Processing Workshop 2022*, pages 130–142. Association for Computational Linguistics, 2022.

- [13] Masha Medvedeva, Michel Vols, and Martijn Wieling. Using machine learning to predict decisions of the European Court of Human Rights. *Artificial Intelligence and Law*, 28(2):237–266, 2020.
- [14] Masha Medvedeva, Martijn Wieling, and Michel Vols. Rethinking the field of automatic prediction of court decisions. *Artificial Intelligence and Law*, 31(1):195–212, 2023.
- [15] Shubham Nigam, Parth Mehta, Arnab Bhattacharya, and Prasenjit Majumder. NyayaAnumana & IndianLegalBench: Towards comprehensive Indian legal judgment prediction and reasoning. *arXiv preprint arXiv:2501.03745*, 2025.
- [16] Shubham Kumar Nigam, Noel Shallum, Kripabandhu Ghosh, and Arnab Bhattacharya. LegalSeg: Unlocking the structure of Indian legal judgments through rhetorical role labeling. *arXiv preprint arXiv:2501.09690*, 2025.
- [17] OpenNyAI. About OpenNyAI. <https://opennyai.org/about>, 2024. Accessed 2024.
- [18] OpenNyAI. OpenNyAI documentation. <https://opennyai.org>, 2024. Accessed 2024.
- [19] T.Y.S.S. Santosh, Vatsal Venkatkrishna, Saksham Argwal, and Matthias Grabmair. Leveraging precedents for legal judgment prediction in low-resource settings: A case study on the European Court of Human Rights. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 3887–3897. ELRA and ICCL, 2024.
- [20] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web – ESWC 2018*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607. Springer, 2018.
- [21] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Peng Cui, Philip S. Yu, and Yanfang Ye. Heterogeneous graph attention network. In *The World Wide Web Conference*, pages 2022–2032. ACM, 2019.

- [22] Haotian Wu, Lingwei Wei, Mingze Li, Wei Zhou, Yan Zhao, Luqing Hou, Dapeng Liu, and Jine Tang. Precedent-enhanced legal judgment prediction with LLM and domain-model collaboration. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12060–12075. Association for Computational Linguistics, 2023.
- [23] Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Chaojun Xiao, Zhiyuan Liu, and Maosong Sun. Legal judgment prediction via topological learning. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3540–3549. Association for Computational Linguistics, 2018.